

An Understanding-First Reinforcement Learning Agent for
Single-Asset Trading:
From-Scratch Policy Gradients and Deep Q-Learning on a
Complete-State Market MDP

Fiyinfoluwa Akano
6962514

School of Computer Science and Electronic Engineering
University of Surrey

September 2026

©2026 Fiyinfoluwa Akano

Statement of Originality

This report and the work to which it refers are the results of my own efforts. Any ideas, data, images or text resulting from the work of others (whether published or unpublished) are fully identified as such within the work and attributed to their originator in the text or bibliography. This report has not been submitted in whole or in part for any other academic degree or professional qualification. I agree that the University has the right to submit my work to the plagiarism detection service TurnitinUK for originality checks.

Fiyinfooluwa Akano

September 2026

Acknowledgements

I thank my supervisor, Mr. Nguyen, whose insistence that every equation be derived before it is implemented — and that every result be checked against the simplest honest alternative — shaped not only this dissertation but the way I now approach technical work. I also thank my family for their patience and support throughout the year.

Abstract

Reinforcement learning (RL) promises trading agents that learn from experience, but markets provide no ground truth against which a broken implementation can be detected. This dissertation adopts an understanding-first methodology: the two canonical RL algorithm families — the Monte-Carlo policy gradient (REINFORCE) and value-based Deep Q-learning — are derived from first principles, implemented from scratch, validated on games with known outcomes, and only then applied to a carefully formulated trading MDP with a complete 9-feature observable state, a divisible-asset action model and a fee-inclusive wealth-change reward. A three-algorithm ablation (adding library PPO) and a fee sensitivity study complete the evaluation, conducted on 26 held-out months of S&P 500 ETF data against properly capitalised baselines. The policy-gradient agent converges to a policy identical to buy-and-hold on every test month — shown to be the rational optimum under a risk-neutral wealth objective — while Deep Q-learning finds a nearby policy with lower drawdown (4.8% vs. 6.0%) and higher monthly Sharpe ratio (0.45 vs. 0.40) at negligible return cost, and reduces its trading intensity threefold as fees rise. PPO proves bimodal across seeds, landing on either rational optimum. A pilot study demonstrates that with a minimal state the collapse to buy-and-hold is universal across policy-gradient optimisers, and that without a fully invested baseline it would be misreported as a ten-fold victory over the market. The dissertation contributes a fully traceable derivation-to-implementation pipeline, an honest evaluation framework for trading RL, and an empirical characterisation of how the algorithm families respond differently to the same objective.

Contents

Statement of Originality	1
Acknowledgements	2
1 Introduction	6
1.1 Motivation	6
1.2 Problem statement	7
1.3 Objectives	7
1.4 Contributions	7
1.5 Scope and non-goals	8
1.6 Dissertation structure and how to read it	8
2 Background & Literature Review	10
2.1 From trial and error to a formal framework	10
2.2 The Markov Decision Process	11
2.3 Value-based methods: from Q-learning to DQN	12
2.4 Policy-gradient methods: REINFORCE and its descendants	12
2.5 Choosing between the families	13
2.6 Reinforcement learning for trading	14
2.7 Partial observability and hidden state	15
2.8 Summary and gap	15
3 Methodology & Mathematical Formulation	17
3.1 Overview: trading as a Markov Decision Process	18
3.2 Episode definition and data regime	18
3.3 The state: what the agent sees	19
3.3.1 Two kinds of information	19
3.3.2 When is a state admissible?	19
3.3.3 The design ladder	20
3.3.4 The state used in this work	21
3.3.5 The risk-aware extension	22
3.3.6 What this representation does not contain	23
3.4 Action space: a divisible asset	24
3.5 Reward function	24

3.6	Objective function	25
3.6.1	Trajectory return	25
3.6.2	Where the parameters live	25
3.7	From the objective to the REINFORCE loss: why the log appears	26
3.7.1	Step 1 — differentiate under the integral (exact)	26
3.7.2	Step 2 — the likelihood-ratio (log-derivative) identity (exact)	26
3.7.3	Step 3 — the unknown market dynamics cancel (exact)	27
3.7.4	Step 4 — causality: reward-to-go (exact in expectation)	27
3.7.5	Step 5 — Monte-Carlo estimation (approximation)	27
3.7.6	Step 6 — the surrogate loss implemented in PyTorch	27
3.8	Deep Q-learning formulation	28
3.9	PPO: the stabilised policy gradient (third ablation row)	28
3.10	Algorithms	29
3.11	Evaluation protocol	30
4	System Design & Implementation	32
4.1	Architecture overview	32
4.2	Data pipeline	33
4.3	The trading environment	34
4.4	From-scratch REINFORCE	35
4.5	From-scratch Deep Q-learning	36
4.6	PPO ablation row	36
4.7	What verification found	37
4.8	Verification and testing	38
4.9	Experimental configuration	38
4.10	Reproducibility	38
5	Empirical Results & Analysis	40
5.1	Experimental setup	40
5.2	Phase 0: algorithm validation on games	40
5.3	Pilot study: the minimal state collapses to buy-and-hold	41
5.4	Main results: the algorithm ablation on the complete state	42
5.5	Fee sensitivity	44
5.6	Preliminary evidence: a risk-aware reward	45
5.7	Analysis and key findings	46
6	Conclusions & Future Work	48
6.1	Achievements against objectives	48
6.2	Principal findings	48
6.3	Critical appraisal and limitations	49
6.4	Future work	49
6.5	Reflection on project management	50
A	Equation-to-Implementation Traceability	52

B	Per-Month Test Results	54
C	Glossary (Plain-Language)	55

List of Figures

3.1	The agent–environment interaction loop for the trading MDP. One cycle corresponds to one trading day.	18
4.1	System architecture: four decoupled modules. The evaluation harness scores every learned agent and every baseline through one shared episode-runner, so no method can gain from a differing evaluation path.	33
4.2	Chronological three-way split of the 96 monthly episodes: 60 training months, 12 validation months and 24 held-out test months.	34
5.1	CartPole-v1: final comparison (left) and learning curves (right, 25-episode moving average). Scratch REINFORCE learns but plateaus below the stabilised methods — the variance limitation predicted by the theory of Section 3.7.	41
5.2	Flappy Bird (left) and LunarLander-v3 (right): every learner clears its random floor under a fixed budget.	42
5.3	Mean test-month wealth change by method (error bars: one standard deviation across months).	43
5.4	Wealth paths on the strongest rising test month (2022-11, left) and the weakest falling month (2023-09, right). REINFORCE overlays buy-and-hold exactly. Deep Q tracks it in the rising month but exits progressively during the decline. PPO (seed 42) stays lightly invested in both.	43
5.5	Exposure (position value as a fraction of C_0) during the falling month 2023-09. Both agents ramp to near-full investment; as the decline develops, Deep Q trims exposure from ≈ 0.9 to ≈ 0.4 while buy-and-hold remains pinned at ≈ 1.0 until forced terminal liquidation.	44
5.6	Training-episode returns (smoothed) for the scratch methods. Month-to-month drift dominates the visible signal; learning progress is measured on held-out months, not on this curve.	45
5.7	Mean ΔW under increasing proportional fees.	46

List of Tables

2.1	The two algorithm families compared in this project.	13
3.1	Notation used throughout Chapters 3–5.	17
3.2	Mapping a popular game MDP to the portfolio trading MDP.	18
3.3	The state-design ladder. Each rung resolves the limitation named in the row above it. ΔP_t is the one-step return, c_t cash and v_t exposure as fractions of initial capital, h_t the units held, τ_t the episode clock and dd_t the fall from peak wealth.	21
3.4	The state $s_t \in \mathbb{R}^9$ used in this work (rung S_3). C_0 is initial cash, h_t the units held, $W_t = C_t + h_t P_t$ the mark-to-market wealth. Windows are $k=5$ for momentum and volatility, $m=20$ for the trend reference and $n=14$ for the average true range.	22
4.1	Hyperparameters and budgets for the final experiments. d is the observation width, 9 for the state of Table 3.4 and 10 for the risk-aware extension.	39
5.1	Phase 0 mean evaluation return (30 greedy episodes, seed 42). CartPole solved ≈ 475 ; LunarLander solved ≈ 200	41
5.2	Pilot (minimal 3-D state, whole shares): all learned policies execute an action sequence identical to fully-invested buy-and-hold on every one of the 26 test months.	41
5.3	Algorithm ablation on 26 held-out SPY test months (seed 42, 80k steps each). ΔW in dollars. Sharpe on monthly returns; MDD on the concatenated test equity curve. “Id. B1b” counts months whose executed action sequence is identical to buy-and-hold. Accounting identity held to $< 10^{-6}$ for every method on every month; the illegal-action rate was zero for all evaluated policies.	42
5.4	Seed spread: mean ΔW in dollars (months identical to buy-and-hold in parentheses). PPO was additionally run on seeds 45–46 to test the bimodality claim.	44
5.5	Fee sensitivity (seed 42, retrained and evaluated per fee level). Mean ΔW in dollars; trades per month in parentheses.	45
5.6	Risk-aware preliminary (seed 42, $\lambda = 0.25$; trained on the shaped reward, evaluated on true wealth metrics).	45
6.1	Project stages and their exit artefacts.	50
A.1	Formulation $\rightarrow$ code $\rightarrow$ verification map.	53
B.1	Per-month ΔW (\$) on the 26 held-out test months.	54

Chapter 1

Introduction

1.1 Motivation

Sequential decision-making under uncertainty is a defining problem of machine intelligence: an agent must act now, on incomplete information, to influence outcomes it will only observe later. Reinforcement learning (RL) is the branch of machine learning built for exactly this setting. Its successes are among the best known in the field — backgammon played at expert level from self-play [1], Atari games mastered from raw pixels [2], and the game of Go, long considered out of reach, won against a world champion [3]. In each case the recipe is the same: an agent tries actions, observes consequences, receives a numerical score, and gradually shifts its behaviour toward whatever earns more of that score.

In everyday terms, reinforcement learning is how one might train a pet: nobody hands the animal a manual, but rewarded behaviour is repeated and unrewarded behaviour fades. The technical content of this dissertation is, at heart, a precise answer to two questions that this analogy leaves open: *what exactly should the learner be shown* (the state), and *what exactly should count as the reward* — because, as our results will demonstrate, the answers determine everything the agent ultimately does.

Financial trading is one of RL’s most natural — and most unforgiving — testbeds. A trading agent observes market and portfolio information, chooses among actions with immediate costs and delayed consequences, and receives an unambiguous scalar verdict: the change in its wealth. The mapping onto the RL formalism is almost mechanical, which is why a large applied literature exists [4, 5, 6].

Trading is also unforgiving for a subtler reason, and this reason shapes the entire methodology of the present work. In games, the correct outcome is known: a CartPole balancer either balances or does not; a Go program either wins or loses. In markets, no oracle labels the right action, so a *broken learning implementation can masquerade as a mediocre trading strategy indefinitely* — there is no error message, only a plausible-looking equity curve. Worse, decades of financial economics [7] warn that beating a simple passive strategy using public price history alone should be rare; an experiment that appears to do so easily is more likely to contain a bug or an unfair comparison than a discovery [8]. This dissertation therefore adopts an *understanding-first* methodology: derive each learning rule from first principles, implement it from scratch, validate it on games with known outcomes, and only then apply it to market data — with baselines designed so that honest conclusions are detectable.

1.2 Problem statement

We study single-asset trading formulated as an episodic Markov Decision Process (MDP): one calendar month of daily prices is one episode; the agent begins each episode holding only cash and must end it in cash. At each trading day it observes a *complete* state — nine numbers summarising the market’s recent behaviour, the time remaining in the episode, and its own balance sheet — and chooses one of three actions: hold, buy a fixed slice of the asset, or sell a slice. Its reward at each step is the change in its total wealth, with realistic transaction fees deducted. The learning problem is to find, from experience alone, the decision rule that maximises expected end-of-month wealth.

Two canonical algorithm families are implemented from scratch and compared under identical conditions: the Monte-Carlo policy gradient (REINFORCE [9]), which learns the decision rule directly, and value-based Deep Q-learning [2], which learns the value of each action and acts greedily on those values. A third row — PPO [10], the stabilised modern policy gradient, via a reference library implementation [11] — completes the algorithm ablation.

1.3 Objectives

- 01** Formulate single-asset trading as an episodic MDP with a complete observable state, a divisible-asset action model, and a wealth-change reward with explicit transaction costs.
- 02** Derive, from first principles, the policy-gradient (REINFORCE) learning rule — including why the implemented loss contains a log-probability term — and the Deep Q-learning rule, and implement both from scratch in PyTorch.
- 03** Validate both implementations on control tasks with known outcomes (CartPole, Flappy Bird, LunarLander) before any market experiment.
- 04** Evaluate both methods on held-out real market data against properly capitalised passive and random baselines, under a chronological split, matched budgets and multiple seeds.
- 05** Analyse the failure and success modes observed — in particular reward-induced policy collapse to buy-and-hold — and articulate the limitations of the formulation, including partial observability and non-IID data.

These objectives were agreed through supervision and deliberately weight *depth of understanding* over breadth of features: a smaller system in which every equation, every line of code and every reported number can be defended, rather than a larger system assembled from library calls whose behaviour cannot be explained.

1.4 Contributions

- A fully derived, from-scratch implementation of REINFORCE and Deep Q-learning for a masked, divisible-asset trading MDP, with every loss line traceable to a stated equation (the traceability map is Appendix A).

- A complete 9-feature state design with explicit rationale, and a pilot study demonstrating empirically why minimal states induce degenerate policies.
- An honest evaluation framework — fully invested buy-and-hold baseline, chronological splits, fee-inclusive rewards, accounting identity verified per episode — under which the central finding is established: under a risk-neutral wealth reward, the policy gradient converges to buy-and-hold exactly, while Deep Q-learning finds a nearby policy with measurably better risk-adjusted characteristics (lower drawdown, higher Sharpe ratio) at negligible return cost.
- A three-algorithm ablation (REINFORCE, Deep Q, PPO) and a fee sensitivity study showing that the value-based agent adapts its trading intensity to transaction costs, while the clipped policy gradient is bimodal across seeds — both behaviours explained mechanistically from the update rules.

1.5 Scope and non-goals

It is as important to state what this project is *not*. It is not an attempt to produce a profitable trading system: the efficient-market null hypothesis [7], the single asset, and the daily-bar granularity all argue against that reading, and Chapter 5 shows why apparent profitability claims in this setting demand suspicion before celebration. It is also not a survey of every RL algorithm: methods are included only where they earn their place in the ablation logic (REINFORCE as the transparent foundation, Deep Q as the structurally different value-based family, PPO as the stabilised descendant). The deliverable is a defensible scientific account: a carefully formulated problem, correctly derived and implemented learners, and an evaluation whose conclusions survive scrutiny.

1.6 Dissertation structure and how to read it

The chapters follow the order in which the work must be understood, and each builds strictly on its predecessor:

- **Chapter 2** reviews the background: what an MDP is, how the two algorithm families work and where they came from, what is known about applying RL to trading, and the evaluation pitfalls this project is designed to avoid.
- **Chapter 3** is the mathematical core: the trading MDP is formulated precisely, and both learning rules are derived step by step from the objective — including the complete answer to why a logarithm appears in the policy-gradient loss.
- **Chapter 4** shows how the mathematics becomes verified code: environment, agents, tests, and the exact hyperparameters used.
- **Chapter 5** reports the experiments in the order the methodology was built: game validation, the pilot collapse study, the main algorithm ablation, and the fee sensitivity study, with analysis of every finding.
- **Chapter 6** closes with achievements against objectives, an honest critical appraisal, future work, and a reflection on project management.

- **Appendices** provide the equation-to-code traceability map (A), the complete per-month results (B), and a plain-language glossary (C).

A reader without a technical background can follow the storyline through the chapter introductions, the plain-language summary that closes each chapter, and the glossary; the mathematics in Chapters 3–4 deepens, but never replaces, that narrative.

Chapter summary (plain language). This project teaches two classic learning algorithms, built from the ground up, to trade a stock fund with imaginary money on real historical prices. Because markets never tell you what the right decision was, the project’s first rule is honesty: prove the learners work on video games first, make them show their working at every step, and compare them against the simplest sensible strategy — just buying and holding. The next chapter explains the ideas this is built on.

Chapter 2

Background & Literature Review

This chapter reviews the ideas this dissertation is built on. It begins with what reinforcement learning is and how it differs from the more familiar kinds of machine learning; formalises the Markov Decision Process, the mathematical object every later chapter manipulates; then traces the two algorithm families compared in this project — value-based methods (Q-learning to DQN) and policy-gradient methods (REINFORCE to PPO) — from their origins to their modern forms. The second half turns to the application domain: what the literature reports about reinforcement learning for trading, the recurring evaluation mistakes that make much of that literature unreliable, and the partial-observability question that markets pose. The chapter closes by stating the gap this project occupies.

2.1 From trial and error to a formal framework

Most machine learning taught at masters level is *supervised*: a model is shown inputs together with correct answers (this image is a cat; this email is spam) and learns to reproduce the labelling. Reinforcement learning [12] removes the answer sheet. The learner — the *agent* — interacts with a world — the *environment* — in a loop: it observes the situation, chooses an action, and receives back two things only: a new situation and a numerical *reward*. Nobody ever tells it which action was correct; it must discover, by trial and error, which behaviour accumulates the most reward over time.

Two consequences of removing the answer sheet drive everything in this dissertation. First, *credit assignment*: when reward arrives late (the month's profit is only known at the month's end), the agent must work out which of its many earlier decisions deserve the credit or the blame. Second, *verification*: with no labels, there is no training-set accuracy to inspect, so a defective implementation produces no error — only quietly wrong behaviour. The first consequence motivates the mathematics of Chapter 3; the second motivates this project's game-first validation discipline (Chapter 4), because games are the rare setting where the correct outcome of learning *is* known.

The field's landmark systems illustrate the paradigm's reach: TD-Gammon learned expert backgammon purely from self-play [1]; DQN reached human-level play across dozens of Atari games from raw pixels [2]; AlphaGo defeated a world champion at Go [3]. All three learned from a scalar score alone — precisely the position a trading agent is in, where the score is money.

2.2 The Markov Decision Process

The standard formalism for sequential decision problems is the Markov Decision Process (MDP), whose foundations trace to Bellman’s dynamic programming [13]; the modern treatment followed throughout this dissertation is Sutton and Barto [12]. An MDP is a tuple

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where $\mathcal{S}$ is the set of *states* the world can be in, $\mathcal{A}$ the set of *actions* available, $P(s' \mid s, a)$ the *transition dynamics* (the probability of landing in state s' after taking action a in state s), R the *reward function*, and $\gamma \in (0, 1]$ a *discount factor* that down-weights distant rewards. The agent’s behaviour is a *policy* $\pi(a \mid s)$: a rule (possibly probabilistic) for choosing actions in each state. The learning goal is to find a policy maximising the expected *return* — the cumulative discounted reward $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$.

The Markov property. The definition requires that the state summarise *all decision-relevant history*: given the present state, the past adds nothing. In plain terms, a chess board satisfies this — one photograph of the board is enough to play on; how the pieces got there is irrelevant. A poorly chosen trading state violates it: if the agent is shown only today’s price change but its profit depends on the price it originally paid, then something decision-relevant is hidden, and the theory’s guarantees quietly lapse. This single observation drives the central design decision of this project — the *complete* state of Section 3.3 — and its most important limitation analysis (Section 2.7).

Value functions and the Bellman equation. Two functions organise the solution space. The state-value function $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ scores how good a situation is under policy π ; the action-value function $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ scores how good an action is in that situation. The optimal action-value function obeys Bellman’s recursion,

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right], \quad (2.1)$$

which reads: the value of acting now is the immediate reward plus the discounted value of acting optimally afterwards. Equation (2.1) is not merely notation — it is the operating principle of the entire value-based family reviewed next, and its sampled, approximate form is the loss function our Deep Q implementation minimises (Section 3.8).

Exploration versus exploitation. A learner that always takes the action it currently believes best may never discover a better one; a learner that experiments forever never banks its knowledge. Every algorithm in this dissertation embodies a stance on this dilemma: value-based methods explore explicitly (taking a random action with probability ε), while policy-gradient methods explore implicitly, by sampling actions from a probabilistic policy that only gradually sharpens.

2.3 Value-based methods: from Q-learning to DQN

The value-based family answers “what should I do?” indirectly: learn how much each action is *worth*, then pick the most valuable. Its lineage begins with temporal-difference (TD) learning [14], which updates value estimates from the difference between successive predictions rather than waiting for final outcomes, and reaches its canonical form in Q-learning [15]: after each transition (s, a, r, s') , update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (2.2)$$

nudging the stored value toward the Bellman target with learning rate α . Remarkably, this converges to Q^* for finite MDPs regardless of how actions were chosen during learning (it is *off-policy*), provided every state–action pair is visited sufficiently often [15].

The table problem. Tabular Q-learning stores one number per state–action pair, which is fine for a small grid-world and hopeless for anything continuous. An intuition used repeatedly in this project’s supervision: imagine a haulage robot whose state is its position in a mine. Discretise the mine coarsely and the table is small but the robot is blind to distinctions that matter; discretise finely and the table outgrows memory — and either way, experience at one location teaches nothing about the location one metre away, because table cells share nothing. Our own Phase-0 experiments reproduce this on CartPole: tabular Q-learning succeeds *only after* the 4-dimensional continuous state is hand-binned, and the approach is a non-starter for the 9-dimensional continuous trading state (Section 5.2).

Function approximation and its dangers. The remedy is to replace the table with a function approximator — a neural network $Q_\phi(s, a)$ — that *generalises* across nearby states. The cost is that convergence guarantees weaken substantially: the combination of function approximation, bootstrapping and off-policy data (the “deadly triad”) can diverge [16, 12]. Deep Q-learning (DQN) [2] demonstrated that two mechanisms restore practical stability: *experience replay* [17] — storing transitions in a buffer and training on random minibatches, which breaks the strong temporal correlation of consecutive experiences — and a periodically frozen *target network*, which stops the regression target from chasing its own updates. Both mechanisms are implemented from scratch in this project (Section 4.5), and both matter doubly for market data, whose consecutive samples are even more correlated than game frames. Refinements such as double Q-learning [18] address residual overestimation bias; they are noted as extensions rather than implemented, to keep the from-scratch implementation minimal and fully understood.

2.4 Policy-gradient methods: REINFORCE and its descendants

The policy-gradient family answers “what should I do?” directly: write the policy as a differentiable function $\pi_\theta(a | s)$ — in practice a neural network whose weights are θ — and adjust θ by gradient ascent on the expected return $J(\theta)$. The foundational algorithm is REINFORCE [9]. Its enabling mathematical step is the likelihood-ratio (score-function) identity $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$, which converts the gradient of an expectation *over* a distribution into an expectation *of* a gradient — something that can be estimated from sampled episodes. This identity is the sole reason a $\log \pi_\theta$ term appears in every policy-gradient loss; the full six-step derivation, each step labelled exact or approximate, is given in Section 3.7.

REINFORCE’s virtue is transparency: it is unbiased and involves no bootstrapping, no replay, and no auxiliary networks. Its vice is *variance*: each gradient estimate is scaled by a full Monte-Carlo return, a noisy quantity, so learning is slow and unstable. The subsequent thirty years of policy-gradient research can be read as a sequence of patches on this one weakness:

- **Baselines and advantage.** Subtracting a state-dependent baseline $b(s)$ from the return leaves the gradient unbiased but can reduce variance dramatically; learning that baseline with a second network (a *critic*) gives actor-critic methods [19], whose modern parallel form is A2C/A3C [20]. Generalised advantage estimation [21] tunes the bias–variance trade-off of the advantage signal itself.
- **Constrained updates.** Even with a good advantage signal, a single large policy update can be destructive, because the data was collected under the *old* policy. Trust-region methods [22] bound the update size formally; Proximal Policy Optimization (PPO) [10] achieves a similar effect by simply *clipping* the update ratio, and has become the default policy-gradient method in applied work, including most trading-RL frameworks [6].

This project retains REINFORCE as the primary policy-gradient method precisely *because* it is the un-observed foundation: every design element of the successors is a response to a weakness REINFORCE exhibits transparently, and our Phase-0 results reproduce those weaknesses on cue (CartPole: REINFORCE 292 vs. PPO 500, Section 5.2). PPO is retained as the ablation’s third row so the patched descendant can be compared with its ancestor on the same problem.

2.5 Choosing between the families

The two families make opposite engineering trade-offs, summarised in Table 2.1. Neither dominates: the value-based route updates from every single step and reuses old experience, but bootstraps (estimates built on estimates) and only *derives* a policy indirectly; the policy-gradient route optimises the actual objective with unbiased gradients, but needs whole episodes per update and discards experience once used. Comparing structurally different learners on the *same* problem is therefore scientifically informative — if both converge to the same behaviour, that behaviour likely reflects the problem’s true optimum rather than an algorithmic artefact. This logic is the core experimental design of Chapter 5.

Table 2.1: The two algorithm families compared in this project.

	Value-based (Deep Q)	Policy-gradient (REINFORCE)
Learns	action values $Q_\phi(s, a)$	the policy $\pi_\theta(a s)$ itself
Update timing	every environment step	once per completed episode
Data reuse	replay buffer (off-policy)	fresh rollouts only (on-policy)
Estimator bias	biased while learning (bootstraps)	unbiased
Estimator variance	low (one-step targets)	high (full Monte-Carlo returns)
Exploration	explicit (ϵ -greedy)	implicit (stochastic policy)
Failure style	divergence/overestimation	noisy, slow convergence

2.6 Reinforcement learning for trading

Applying RL to trading is an old idea with a well-documented set of pitfalls. The foundational work is Moody and Saffell’s *direct reinforcement* [4], which trains a trading policy directly by gradient methods and — presciently for this project — argues that the *objective* must be risk-adjusted (they propose a differential Sharpe ratio), because maximising raw profit incentivises maximum exposure. That is an early recognition of the theme our results confirm empirically: *the reward function defines the incentive*, and an agent can only be blamed for optimising what it was actually asked to optimise. Deng et al. [5] extended direct RL with deep recurrent representations; Jiang et al. [23] treat multi-asset portfolio management with deep RL on cryptocurrency data. Modern open frameworks, notably FinRL [6], standardise Gym-style market environments and default to PPO-family agents — convenient, but, as this dissertation argues, convenience without derivation invites exactly the silent failures that motivated our from-scratch requirement.

A close reading of the nearest prior work. The study closest to ours is Théate and Ernst [24], who benchmark a trading DQN (TDQN) against buy-and-hold and classic active strategies across 30 instruments. Read critically, their own results anticipate this dissertation’s central finding. Their headline is that TDQN “outperforms benchmark strategies on average” — but the strategies it beats on average are the *active* ones (trend-following posts a *negative* average Sharpe on their testbench), while, in their own words, TDQN “only barely surpasses the buy and hold strategy on these particular bullish markets.” Most tellingly, on the S&P 500 ETF — the very asset studied here — their reported test Sharpe for TDQN is 0.834, *identical* to buy-and-hold’s 0.834; they observe that TDQN’s performance “is identical or very close to the passive trading strategies for multiple stocks” because the agent “efficiently learns to tend toward a passive trading strategy.” They further report substantial run-to-run variance and overfitting on volatile single names (Tesla). In short: a careful prior study, honestly read, already shows value-based RL converging toward passivity on index products under profit-seeking objectives — a result their framing treats as a footnote, and this dissertation places at the centre, explains mechanistically, and reproduces with fully derived implementations.

Evaluation pitfalls. Surveys of the area [25] and the financial machine-learning methodology literature [8, 26] repeatedly flag failure modes that this project’s design guards against explicitly:

1. **Weak baselines.** Much published work beats only strawman alternatives. The baseline that matters is buy-and-hold, *properly capitalised* — deploying the same capital the agent may deploy. Section 5.3 shows a live example where an unfair baseline would have converted a null result into a spurious ten-fold victory.
2. **Look-ahead leakage.** Shuffling time-series data before splitting lets fragments of the future into training. All splits in this project are strictly chronological (Figure 4.2).
3. **Ignored transaction costs.** Costless backtests reward hyperactive trading that fees would destroy. Here, fees live *inside* the reward, so the agent pays them during learning — and Section 5.5 shows the value-based agent measurably adapting its behaviour to them.
4. **Backtest overfitting.** Repeatedly tuning on the same held-out period converts test data into training data [8]. Our defence is procedural: fixed budgets and hyperparameters chosen on games, one

evaluation protocol declared in advance, multiple seeds reported.

The efficient-market null hypothesis. Financial economics supplies the prior against which any trading result must be read: in a broadly efficient market [7], persistent excess return over buy-and-hold from public price history alone should not be expected, and the risk–return trade-off [27] implies that reducing risk, not only raising return, is a legitimate form of success (measured here by the Sharpe ratio [28] and maximum drawdown). This dissertation therefore treats “the agent rediscovers buy-and-hold” as a scientifically meaningful finding rather than a failure — and treats any easy-looking victory as a signal to audit the experiment, not to celebrate.

2.7 Partial observability and hidden state

The MDP formalism assumes the agent sees a state satisfying the Markov property. When decision-relevant structure is hidden — as it plainly is in markets, where order flow, news, and the strategies of other participants never appear in a price series — the problem is formally a *Partially Observable* MDP (POMDP) [29]. The principled treatment maintains a *belief state*: a probability distribution over what the hidden variables might be, updated as observations arrive. The closely related hidden Markov model [30] formalises the special case where an unobserved discrete regime (e.g. “calm market” vs. “turbulent market”) generates the observations — a natural reading of financial data, and one raised explicitly in this project’s supervision as the honest way to think about what a price-only state misses.

Exact POMDP solutions are intractable for all but tiny problems [29]; practical systems either learn a memory (recurrent policies) or — the route taken here — *engineer the observation to be as complete as practically possible* and state the residual gap honestly. Chapter 3 designs a state that includes everything the agent can legitimately know (market summary statistics, the episode clock, and its full balance sheet); Chapter 6 returns to the POMDP view as the principal direction for future work.

2.8 Summary and gap

The literature offers, on one side, two mature algorithm families whose mechanics and trade-offs are well understood on games (Sections 2.3–2.5); on the other, a trading-RL corpus of genuine ideas [4, 5, 24] interleaved with results weakened by the evaluation pitfalls of Section 2.6. Between them sits a gap that is methodological rather than algorithmic: *few works formulate one careful trading MDP and subject structurally different, fully understood learners to an evaluation designed to make honest conclusions detectable*. That is the gap this dissertation occupies: a single formulation (Chapter 3); from-scratch implementations whose every line is traceable to a derived equation (Chapter 4, Appendix A); validation on known-outcome games first; and an evaluation against properly capitalised baselines under chronological splits, fee-inclusive rewards and multiple seeds (Chapter 5).

Chapter summary (plain language). Reinforcement learning means learning by trial and error from a numerical score. Two big families exist: one learns *how much each choice is worth* and picks the most valuable; the other learns *the choosing rule itself* and nudges it toward whatever scored well. People have applied both to trading for decades, but published results are often flattered by unfair comparisons — weak opponents, peeking at the future, ignoring trading costs. Markets also hide information no price chart

contains, which the theory calls partial observability. This project's plan follows directly: build one honest trading problem, implement both families from the ground up, prove them on games first, and compare them fairly. The next chapter builds that problem precisely.

Chapter 3

Methodology & Mathematical Formulation

This chapter formulates single-asset trading as an episodic Markov Decision Process (MDP). It first sets out the state the agent observes, tracing how that state was arrived at through three successive versions and stating precisely how far it can be called complete. It then derives, from first principles, the two priority learning rules of this project — the Monte-Carlo policy gradient (REINFORCE) and Deep Q-learning — and introduces PPO as a third, stabilised policy-gradient ablation row. Particular care is taken to show *why* the implemented loss functions follow from the stated objective — i.e. why the policy-gradient loss contains a $\log \pi_\theta$ term, and whether each step is exact or a sampling approximation. The chapter closes with the evaluation protocol used in Chapter 5.

In plain terms, this chapter answers four questions precisely: what the agent *sees* (the state), what it may *do* (the actions), what it is *paid* (the reward), and *how it learns* (the update rules). Every symbol used is collected in Table 3.1 for reference.

Table 3.1: Notation used throughout Chapters 3–5.

Symbol	Meaning
$s_t \in \mathbb{R}^9$	state observed at step t (Table 3.4)
$a_t \in \{0, 1, 2\}$	action: Hold, Buy slice, Sell slice
r_t	reward: one-step wealth change, net of fees
P_t	asset price at day t ; T : episode length (days)
C_t, h_t	cash balance; (fractional) units of the asset held
$W_t = C_t + h_t P_t$	mark-to-market wealth; C_0 : initial cash
c_t, v_t	cash and exposure as fractions of C_0
dd_t	fall from peak wealth within the episode (rung S_4)
c	proportional fee rate (5 basis points = 0.05%)
$\Delta = 0.1 C_0$	fixed monetary trade slice
λ	weight on the drawdown penalty in the risk-aware reward
γ	discount factor
τ	a trajectory (one episode); also the clock feature t/T
$G(\tau), G_t$	trajectory return; reward-to-go from step t
$\pi_\theta(a s)$	policy network with parameters θ
$Q_\phi(s, a), Q_\phi^-$	Q-network with parameters ϕ ; frozen target copy
$J(\theta)$	objective: expected trajectory return
ε	exploration rate (ε -greedy)

3.1 Overview: trading as a Markov Decision Process

Reinforcement Learning (RL) trains an agent to make sequential decisions by interacting with an environment. The environment is formalised as a discrete-time MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: at step t the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives a scalar reward r_t , and the environment transitions to s_{t+1} according to dynamics $P(s_{t+1} | s_t, a_t)$. Figure 3.1 shows this interaction loop instantiated for the trading task.

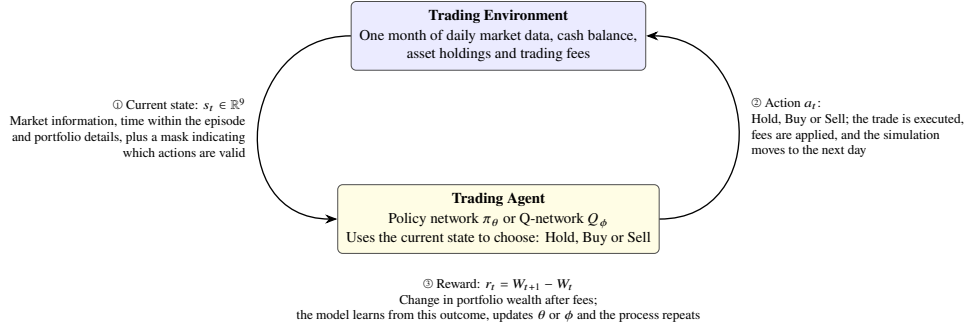


Figure 3.1: The agent–environment interaction loop for the trading MDP. One cycle corresponds to one trading day.

To keep the formulation concrete, Table 3.2 maps the components of a familiar game-playing task (Flappy Bird, used in our Phase-0 validation) onto the trading task.

Table 3.2: Mapping a popular game MDP to the portfolio trading MDP.

MDP component	Flappy Bird	Trading (this work)
State s_t	Bird’s position and speed, together with the position of the next pipe	Market information and the agent’s current portfolio position
Action a_t	Flap or do not flap	Hold, Buy, or Sell
Reward r_t	Survival score based on how long the bird remains in play	Change in portfolio wealth after trading fees
Episode τ	One complete game session	One calendar month of daily trading
Terminal rule	The game ends when the bird crashes or hits the ground	The episode ends at the final trading day, with any remaining position sold

The analogy also fixes the design principle used throughout. In Flappy Bird the state sees the whole of the world that matters to the decision, namely the bird’s motion and the geometry of the next pipe, and an agent given that state need never guess. The trading state is held to the same test. Section 3.3 applies it, and shows that the test can be met in full for one half of the trading world and only approximately for the other.

3.2 Episode definition and data regime

Episode. One episode is one calendar month of daily closing prices $P_0, P_1, \dots, P_T$ (with $T \approx 18\text{--}23$ trading days per month). The agent starts each episode with cash only and is force-liquidated at the final

bar, so no position is carried between episodes.

Independence assumption. Episodes are treated as mutually independent, exactly as successive game sessions are independent. This is a modelling simplification: consecutive months of market data are in fact correlated (the data are non-IID in time). Handling that temporal dependence properly belongs to time-series modelling and partially observable formulations, and is deliberately out of scope; the assumption and its consequences are revisited in the conclusions.

Why monthly episodes of daily bars? A single year of data yields only about 12 monthly episodes but roughly 250 daily transitions. Eight years (2018–2025) provide 96 months and about 2,000 transitions, split chronologically with no shuffling, so no test information can leak backwards (Section 3.11). Minute-bar episodes ($T=390$ per session) are an immediate drop-in upgrade because the environment interface does not depend on the horizon; this is noted as future work.

3.3 The state: what the agent sees

3.3.1 Two kinds of information

The design principle stated in Section 3.1 is that the state should see the whole of the world that matters to the decision, as the Flappy Bird state does. Before building such a state it is worth noticing that a trading agent’s world has two halves, and that they behave very differently.

The first half is the agent’s own account: how much cash it holds, how much of the asset it owns, what it paid for that position, and how far the month has run. The agent controls this half, and none of it is hidden or estimated. Given the price path, the account’s evolution under any sequence of actions follows exactly from these quantities, by arithmetic the environment performs in closed form. For this half, complete observation is not an aspiration but a property that can be checked.

The second half is the market. Here complete observation is impossible in principle rather than merely inconvenient. Prices are set by the orders, information and intentions of every other participant, and none of that appears in a series of closing prices. However many features are computed from past prices, the future remains only partly predictable from them. Formally the trading problem is a partially observable MDP [29]; read from the other direction, it is the classical observation that prices already reflect the information available about them [7].

The Flappy Bird analogy therefore needs one qualification, and it is better to state it than to be asked for it. Flappy Bird’s world is finite and deterministic, so a handful of numbers genuinely is sufficient. A market’s world is neither. The attainable goal is a state that is complete on the account side and rich enough on the market side that adding further features stops changing what the agent does. That second condition is an empirical claim, and Section 3.3.6 is explicit about how far the present work can support it.

3.3.2 When is a state admissible?

It is tempting to design a state by asking which features look useful. That question is too weak, because it tests only whether the agent can choose sensibly and says nothing about whether the object being trained is an MDP at all. Three conditions are required, and they are checked in order.

1. **Reward measurability.** The reward must be a function of the state, the action and the successor state. If it depends on anything else, the agent is being scored on information it cannot see.
2. **Transition closure.** The successor state must be computable from the current state, the action and the exogenous price, without reaching back into history.
3. **Decision sufficiency.** The state must let the agent distinguish situations that call for different actions.

Only the third is about usefulness. The first two decide whether the formulation is legitimate, and they are the reason two apparently similar quantities are treated very differently below.

Checking the first condition for this work. The reward of Section 3.5 is the wealth change $W_{t+1} - W_t$. Since $W_t = C_t + h_t P_t$, the cash and exposure features of Table 3.4 satisfy $c_t + v_t = W_t/C_0$, and therefore

$$r_t = C_0[(c_{t+1} + v_{t+1}) - (c_t + v_t)]. \quad (3.1)$$

The reward is a function of (s_t, a_t, s_{t+1}) with nothing hidden, so the first condition holds. This is a small calculation but not an idle one: it is what licenses the derivations later in this chapter.

Two kinds of redundancy. Equation (3.1) also shows why the episode's running profit, $W_t/C_0 - 1$, is deliberately *not* a feature. It equals $c_t + v_t - 1$ exactly, so it is recoverable *pointwise*, from quantities the agent already observes at the same instant, and a single linear layer can form it. Supplying it would add width without adding information.

Drawdown behaves differently. Write $D_t = \max_{u \leq t} W_u - W_t$ for the fall from the highest wealth reached so far. Two episodes can arrive at identical cash, identical exposure and identical wealth by different routes: one that rose steadily to its present value and one that reached a much higher peak and gave most of it back. The second is in a materially different position, and no combination of current-step features distinguishes the two, because a running maximum is a property of the *path*. Drawdown is therefore not redundant; it is simply absent unless it is supplied.

This gives a rule that decides such cases without appeal to taste: *exclude features that are pointwise recoverable, and supply features that are path-dependent and relevant*. It is applied twice in what follows.

3.3.3 The design ladder

The state was arrived at by starting from the smallest representation in which a trading decision can be expressed and asking, at each stage, what situations the agent still cannot tell apart. Each rung is therefore justified by a limitation of the one before it rather than by a wish for more inputs. The progression is one of increasing informational completeness, not of increasing feature count.

Table 3.3 sets out the ladder. It is a design argument, and it is stated here before any experiment: Chapter 5 tests whether each addition actually changes what is learned, and the two questions are kept apart deliberately, so that a reasoned specification is not confused with evidence about its consequences.

Two remarks on how to read the ladder. First, the step from S_1 to S_2 is a replacement rather than an addition: the raw unit count gives way to the position's value together with its unrealised return. Moody and Saffell make the underlying point in arguing that a trading system must be told its own position, and

Table 3.3: The state-design ladder. Each rung resolves the limitation named in the row above it. ΔP_t is the one-step return, c_t cash and v_t exposure as fractions of initial capital, h_t the units held, τ_t the episode clock and dd_t the fall from peak wealth.

Rung	State	Limitation that motivates the next rung
S_0	$[\Delta P_t]$	The agent sees the market but not itself. It cannot tell whether it holds a position, or whether a Buy is affordable, so the same market observation covers situations needing opposite actions.
S_1	$[\Delta P_t, c_t, h_t]$	It knows how many units it holds but not what they are worth. Ten units is a small position at one price and a large one at another, and a feed-forward network cannot form the product $h_t P_t$ reliably from its inputs.
S_2	$[\Delta P_t, c_t, v_t, \text{PnL}_t]$	It can value its position and tell profit from loss, but not where it stands in the month. A Buy on the first day and a Buy the day before forced liquidation are different decisions that look identical.
S_{3a}	$[\Delta P_t, c_t, v_t, \text{PnL}_t, \tau_t]$	Its own circumstances are now fully described, but its view of the market is still a single lagged return. It cannot tell a sustained trend from a reversal, or a calm day from a turbulent one.
S_3	adds trend, volatility, distance from trend, and daily range (Table 3.4)	Complete with respect to the environment as defined, but with no memory of the path by which its wealth was reached.
S_4	adds dd_t	Required if the reward refers to risk history (Section 3.3.5).

told it in a form it can use [4]. Second, the rungs are cumulative from S_2 onward, so any difference in behaviour between neighbouring rungs is attributable to the information added between them.

3.3.4 The state used in this work

Rung S_3 is the representation carried into the experiments. It contains nine features in four blocks: what the *market* is doing, what the agent’s *portfolio* holds, how the current *position* is performing, and how much *time* remains. Every feature is scale-free, being a return, a ratio, or a fraction of initial capital, so the network never sees a raw dollar price. This matters because the price of the traded instrument roughly doubles over the sample, and a network fed absolute prices would face inputs from training that never recur at test time.

Why the daily range earns its place. Features 1 to 4 are all functions of the closing-price series. Feature 5 is not, and that is precisely the argument for it. Consider two days that open and close at the same price: one drifts within a narrow band, the other falls sharply and recovers by the close. Their closing returns are identical, their momentum and volatility histories are identical, and any indicator computed from closes alone must treat them as the same day. They are not the same day, and the difference is one of risk. Because the average true range is built from the daily high and low [31], it supplies information that no

Table 3.4: The state $s_t \in \mathbb{R}^9$ used in this work (rung S_3). C_0 is initial cash, h_t the units held, $W_t = C_t + h_t P_t$ the mark-to-market wealth. Windows are $k=5$ for momentum and volatility, $m=20$ for the trend reference and $n=14$ for the average true range.

#	Feature	Definition	What it tells the agent
<i>Market</i>			
1	ΔP_t	$P_t/P_{t-1} - 1$	the immediate price movement
2	mom_t	$P_t/P_{t-k} - 1$	whether that movement is part of a trend
3	vol_t	std. of the last k one-step returns	how variable the market currently is
4	gap_t	$P_t/\text{MA}_m(P) - 1$	how far price sits from its recent average
5	rng_t	$\text{ATR}_n(P)/P_t$	how wide the daily range is, which closing prices cannot show
<i>Portfolio</i>			
6	c_t	C_t/C_0	how much cash is available to deploy
7	v_t	$h_t P_t/C_0$	what the open position is worth
<i>Position</i>			
8	PnL_t	$(P_t - \bar{P}_t^{\text{entry}})/\bar{P}_t^{\text{entry}}$	whether the position is in profit or loss
<i>Time</i>			
9	τ_t	$t/T \in [0, 1]$	how long remains before forced liquidation

re-encoding of closing prices can recover. This distinction, between a feature that adds information and one that merely re-expresses what is already present, is the test used throughout: a candidate feature is rejected if it is a function of data the agent already observes.

A note on how the features are computed. All market features are calculated on the continuous price series and only then divided into monthly episodes. Computing a rolling window inside an episode would leave the early days of every month with no history to average over, so the agent would begin each month believing the market had no trend and no volatility. Momentum does not reset because a new month has started. The implementation and its verification are described in Section 4.8.

3.3.5 The risk-aware extension

Rung S_4 adds a single feature,

$$\text{dd}_t = \frac{\max_{u \leq t} W_u - W_t}{\max_{u \leq t} W_u}, \quad (3.2)$$

the fraction of its peak wealth the agent has given back within the current episode. It is kept out of the main state and introduced as an extension for a reason that is methodological rather than practical. Under the wealth-change reward, drawdown is not needed: the first admissibility condition is already satisfied by (3.1), and adding the feature would enlarge the observation to serve a reward that is not being used. Folding it in would also make its effect impossible to measure, since there would be no comparison in which it was absent. Separating it converts an assumption into a question that Chapter 5 can answer: does explicitly representing risk history change what the agent learns?

The extension becomes *necessary*, rather than optional, as soon as the reward refers to risk. A reward

of the form

$$r'_t = \Delta W_t - \lambda C_0 \max(0, dd_{t+1} - dd_t) \quad (3.3)$$

depends on peak wealth, which S_3 does not contain. Pairing (3.3) with S_3 would violate the first admissibility condition, and the resulting object would not be an MDP, so the derivations in this chapter would not apply to it. That combination is therefore excluded from the experimental design rather than run and explained afterwards; Section 3.11 states which combinations are admissible. The environment enforces the constraint at construction, so an inadmissible configuration cannot be built by accident.

3.3.6 What this representation does not contain

By the division of Section 3.3.1, the portfolio, position and time blocks are complete. Together with the price path, they determine the account's evolution under any sequence of actions exactly, and the accounting identity test of Section 3.11 verifies this numerically in every episode. The market block is an approximation, and the honest formulation of the claim is therefore that s_t is complete *with respect to the trading environment as defined*, and is a fully observable approximation of a market that is not.

The specific omissions are worth naming. Trend and volatility are measured over one window length, so a change of regime that shows up as a shift between short-horizon and long-horizon variability is not directly represented. Traded volume, which carries information about the conviction behind a price move, is available in the data but excluded from S_3 ; it is examined as a further rung rather than assumed to help. Nothing outside the traded instrument enters the state, so forward-looking measures such as implied volatility, and any cross-asset or macroeconomic context, are unavailable.

Other quantities cannot be added at all. Order-book depth and order flow, the text of news as it arrives, the positions and intentions of other participants, and the latent state of the market are unobserved by construction rather than by omission. Théate and Ernst attribute the overfitting in their own study to an observation space that was too limited [24], which is a reminder that this boundary has practical consequences and not merely formal ones. The standard remedies, recurrent policies or an explicit belief state, lie outside the scope agreed for this project and are set out as future work in Chapter 6.

Two scope decisions also bound what the state could usefully contain. The agent trades a single instrument, whereas a portfolio in the sense of Markowitz [27] requires a vector of weights across assets and an estimate of how they move together. The agent is also long-only, so it may decline to participate but cannot express a negative view.

Finally, extending a state is not free. The training period supplies 60 months, which is roughly 1,250 daily transitions. Every additional input increases the number of parameters to be estimated from that fixed and rather small sample, raising the variance of what is learned and the risk of fitting one particular history rather than the process behind it [26]; Bellman's observation about the cost of dimension applies directly [13]. A state should therefore be extended only while further features continue to change measured behaviour, which is why the ladder above is accompanied by the stopping rule declared in Section 3.11 rather than left to judgement after the fact.

3.4 Action space: a divisible asset

The action set is fixed and discrete:

$$\mathcal{A} = \{ 0 : \text{Hold}, \quad 1 : \text{Buy}, \quad 2 : \text{Sell} \}.$$

Action representation is a design choice, not a detail. The state is not the only thing that has to be represented well. Under a whole-share model, “Buy one share” commits a different amount of money at every price level, so the economic meaning of the action drifts across several years of data, and the action becomes infeasible whenever cash falls below one share price. The same label then denotes a small position early in the sample and a large one late in it, which is a representational defect rather than a market fact.

We therefore treat the asset as divisible, as with a fractional-share broker or a cryptocurrency: a Buy converts a fixed monetary slice $\Delta = 0.1 C_0$ of cash into asset units at the current price, and a Sell liquidates up to the same slice of position value. Ten consecutive Buys move the portfolio from all-cash to approximately fully invested, and the meaning of one action is constant throughout.

Because state design and action design are separate representational axes, changing both at once would make any difference in behaviour impossible to attribute. The action model is therefore fixed at $\Delta = 0.1 C_0$ before the agents are compared, and the whole-share alternative is retained as a single controlled comparison at the frozen state (Section 3.1.1) rather than crossed with every rung of the ladder.

Execution and fees. With proportional fee rate c (5 basis points), a Buy spending x dollars acquires $x/(P_t(1+c))$ units, and a Sell of u units returns $uP_t(1-c)$ dollars.

Feasibility masking. Buy requires cash above a minimum trade value; Sell requires a position above the same minimum. Infeasible actions are masked: during learning and evaluation their probabilities (REINFORCE) or values (Deep Q) are set to $-\infty$ before the action is chosen, so illegal trades are never executed. The environment additionally forces Hold if an illegal action is ever proposed, and counts such proposals as a standing diagnostic.

Terminal rule. At the final bar the environment liquidates any remaining position at the closing price (fee included), so every episode ends in cash and episode wealth is unambiguous.

3.5 Reward function

The reward is the one-step change in mark-to-market wealth, net of fees:

$$r_t = W_{t+1} - W_t, \quad W_t = C_t + h_t P_t. \quad (3.4)$$

Three worked cases make the incentive explicit. Holding cash only: $r_t = 0$. Holding h units through a price move: $r_t = h(P_{t+1} - P_t)$. Executing a trade of x dollars: the fee cx is paid immediately inside the wealth change, so every unnecessary round-trip costs $\approx 2cx$. Because rewards telescope, $\sum_{t=0}^{T-1} r_t = W_T - W_0$: maximising the sum of step rewards is exactly maximising final wealth. This identity is verified numerically in every experiment (the “accounting identity” test).

A fully numeric example. Let $C_0 = \$10,000$, price $P_t = \$500$, fee $c = 0.0005$. The agent Buys one slice ($\Delta = \$1,000$): it acquires $1000/(500 \times 1.0005) = 1.999$ units, and its wealth drops immediately by the fee's worth, $r_t \approx -\$0.50$. Overnight the price rises 1% to $\$505$: holding 1.999 units, $r_{t+1} = 1.999 \times \$5 \approx +\10.00 . If it then Sells the slice at $\$505$, it receives $1.999 \times 505 \times 0.9995 \approx \$1,009$, paying another $\approx \$0.50$ fee. Net effect of the round trip: about $+\$9.00$ — the $\$10$ price gain minus $\$1$ of fees. The same trip on a flat price loses $\$1$. The agent is never told any of this; it must discover from such rewards that trading has a price and only pays when the market moves favourably.

On the choice of a risk-neutral reward. Equation (3.4) pays for expected wealth and is silent about risk: a policy that earns $\$100$ steadily and one that oscillates wildly around the same mean earn identical total reward. This is a deliberate, supervision-agreed scoping decision — it keeps the objective simple enough to derive and audit end-to-end — but it has a predictable consequence, developed in Chapter 5: whenever the market drifts upward on average, *maximum exposure is optimal*, and a rational learner should discover buy-and-hold. Risk-aware alternatives (e.g. $r_t = \Delta W_t - \lambda D_t$ with a drawdown penalty, or the differential Sharpe ratio of Moody and Saffell [4]) change that incentive and are analysed as future work in Chapter 6.

A constraint linking the reward to the state. Changing the reward is not a free choice, because the two must remain consistent. Any quantity the reward depends on must be observable, or the first admissibility condition of Section 3.3.2 fails and the learning rules derived below no longer apply. The drawdown-penalised reward (3.3) is exactly such a case, which is why it is paired only with the extended state S_4 and never with S_3 .

3.6 Objective function

3.6.1 Trajectory return

A trajectory (episode) is $\tau = (s_0, a_0, r_0, s_1, \dots, s_T)$, and its discounted return is

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t, \quad \gamma \in (0, 1]. \quad (3.5)$$

3.6.2 Where the parameters live

The agent's decision rule is a *policy network* $\pi_\theta(a \mid s)$: a neural network with weights θ that maps a state to a probability distribution over the three actions. The weights θ parameterize the *policy* — not the reward. The reward function (3.4) is fixed by market mechanics; θ influences the collected rewards only *indirectly*, by changing which actions are sampled and therefore which trajectories occur. Formally, the probability of a trajectory under policy π_θ is

$$p_\theta(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t), \quad (3.6)$$

where $\rho(s_0)$ is the initial-state distribution and P the (unknown) market dynamics. The objective is the *expected* return under this distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [G(\tau)], \quad \theta^* = \arg \max_{\theta} J(\theta). \quad (3.7)$$

Note that θ enters (3.7) only through the sampling distribution p_θ . If it did not — if nothing in the expression depended on θ — the objective would be a constant and there would be nothing to optimise.

In plain terms: a single month’s profit is luck as much as skill, so we cannot judge a policy by one episode. The objective $J(\theta)$ is the *average* profit the policy would earn across all the months it could face, weighted by how likely each is. Changing the network weights θ changes which decisions get made, which changes which futures occur, which changes the average — that indirect chain is exactly what the expectation notation encodes, and what the next section learns to differentiate.

On the discount factor. We use $\gamma = 0.99$. With episodes of only $T \approx 20$ steps, discounting is nearly neutral ($0.99^{20} \approx 0.82$), so the agent is effectively asked to maximise undiscounted end-of-month wealth; $\gamma < 1$ is retained for numerical conditioning and consistency with the standard formulation [12].

3.7 From the objective to the REINFORCE loss: why the log appears

This section answers, step by step, how the implemented loss $L(\theta) = -\sum_t \log \pi_\theta(a_t | s_t) G_t$ follows from (3.7).

3.7.1 Step 1 — differentiate under the integral (exact)

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau) G(\tau) d\tau = \int \nabla_\theta p_\theta(\tau) G(\tau) d\tau. \quad (3.8)$$

This cannot be estimated from samples yet: $\nabla_\theta p_\theta(\tau)$ is not an expectation under p_θ .

3.7.2 Step 2 — the likelihood-ratio (log-derivative) identity (exact)

For any positive function, calculus gives $\nabla_\theta \log p_\theta = \nabla_\theta p_\theta / p_\theta$, hence

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau). \quad (3.9)$$

This identity is the entire reason a logarithm appears in the loss. Substituting (3.9):

$$\nabla_\theta J(\theta) = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) G(\tau) d\tau = \mathbb{E}_{\tau \sim p_\theta} [\nabla_\theta \log p_\theta(\tau) G(\tau)], \quad (3.10)$$

which *is* an expectation under p_θ and can therefore be estimated by sampling trajectories from the current policy.

3.7.3 Step 3 — the unknown market dynamics cancel (exact)

Taking the log of (3.6),

$$\log p_\theta(\tau) = \log \rho(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t),$$

and only the middle sum depends on θ . The gradient of the dynamics terms is zero — which is why the method needs *no model of the market*:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G(\tau) \right]. \quad (3.11)$$

3.7.4 Step 4 — causality: reward-to-go (exact in expectation)

An action at time t cannot influence rewards earned before t , and the corresponding cross-terms have zero expectation. The argument in brief: for $t' < t$, the term $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t | s_t) r_{t'}]$ factorises by conditioning on the history up to t ; the inner expectation $\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t)] = \nabla_\theta \sum_a \pi_\theta(a | s_t) = \nabla_\theta 1 = 0$, because probabilities sum to one whatever θ is (see [12], ch. 13, for the full treatment). Replacing $G(\tau)$ by the reward-to-go $G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$, therefore leaves the gradient unbiased while reducing its variance:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right]. \quad (3.12)$$

3.7.5 Step 5 — Monte-Carlo estimation (approximation)

The expectation is intractable, so it is approximated by the empirical mean over N sampled trajectories — this is the only approximation in the derivation, and it is unbiased:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^{T_n-1} \nabla_\theta \log \pi_\theta(a_t^{(n)} | s_t^{(n)}) G_t^{(n)}. \quad (3.13)$$

The double sum — over trajectories n and over steps t within each trajectory — and the $1/N$ factor are precisely the empirical version of the expectation in (3.7).

3.7.6 Step 6 — the surrogate loss implemented in PyTorch

Automatic differentiation frameworks minimise. Defining

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^N \sum_{t=0}^{T_n-1} \log \pi_\theta(a_t^{(n)} | s_t^{(n)}) G_t^{(n)}, \quad (3.14)$$

where the returns $G_t^{(n)}$ are treated as constants (no gradient flows through them), gives $\nabla_\theta L(\theta) = -\nabla_\theta J(\theta)$ evaluated on the sample. Hence `loss.backward()`; `optimizer.step()` performs stochastic gradient *ascent* on J . The minus sign converts maximisation into the minimisation PyTorch performs; the log is not a modelling choice but the direct consequence of the likelihood-ratio identity in Step 2.

In the implementation the returns within each episode are standardised (zero mean, unit variance) before entering (3.14); this is a standard variance-reduction baseline that leaves the gradient direction unbiased.

3.8 Deep Q-learning formulation

Q-learning is the canonical *value-based* alternative: instead of parameterizing the policy directly, it learns the action-value function. In plain terms, where REINFORCE learns a *habit* (“in situations like this, I tend to buy”), Q-learning learns a *price list* (“in this situation, buying is worth \$3, holding \$1, selling −\$2”) and then simply takes the best-priced action. The price list obeys a self-consistency condition first written down by Bellman [13]:

$$Q^*(s, a) = \mathbb{E} \left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right], \quad (3.15)$$

the expected return of taking action a in state s and acting optimally thereafter. The greedy policy $a = \arg \max_a Q^*(s, a)$ is then optimal.

Why not a table. Tabular Q-learning stores one value per state–action pair. Our state is continuous in nine dimensions; a table over any reasonable discretization explodes combinatorially (and discretization itself throws away information). Deep Q-learning (DQN) replaces the table with a neural network $Q_\phi(s, a)$ that generalises across nearby states.

Learning rule. Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer; minibatches are sampled uniformly to break the strong temporal correlation of consecutive steps. The temporal-difference target uses a separate, periodically synchronised *target network* Q_{ϕ^-} for stability:

$$y_t = r_t + \gamma (1 - d_t) \max_{a' \in \mathcal{A}_{\text{legal}}(s_{t+1})} Q_{\phi^-}(s_{t+1}, a'), \quad L(\phi) = \frac{1}{B} \sum_{b=1}^B (Q_\phi(s_b, a_b) - y_b)^2, \quad (3.16)$$

where d_t indicates episode termination and the max ranges over *legal* actions only (feasibility masking applied inside the target). Exploration uses ε -greedy action selection over legal actions, with ε annealed linearly.

Contrast with REINFORCE. REINFORCE updates once per episode from complete returns (unbiased, high variance, on-policy); DQN updates every step from bootstrapped one-step targets (biased during learning, low variance, off-policy via replay). Comparing the two families on the same MDP is the core experimental design of Chapter 5.

3.9 PPO: the stabilised policy gradient (third ablation row)

REINFORCE and Deep Q are the two priority methods of this project; PPO is included as a third ablation row because it is the standard modern policy-gradient method, and because its design is a direct response

to a weakness REINFORCE exhibits transparently in our own Phase-0 results (CartPole: REINFORCE 292 vs. PPO 500).

Two patches take REINFORCE to PPO. First, an *advantage* estimate replaces the raw return: a learned critic $V_\phi(s)$ provides a baseline, and $A_t = G_t - V_\phi(s_t)$ rescales each gradient term by how much better the action was than expected, reducing variance without adding bias. Second, the policy update is *clipped*. Writing the probability ratio between the new and old policies as

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad (3.17)$$

PPO maximises the clipped surrogate

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad (3.18)$$

with $\epsilon = 0.2$. The min with the clipped term removes any incentive to move the policy ratio outside $[1 - \epsilon, 1 + \epsilon]$ in a single update, preventing the destructively large steps that raw policy gradients permit. Note that L^{CLIP} still contains π_θ through the ratio ρ_t — the same likelihood-ratio machinery of Section 3.7, evaluated against a frozen snapshot $\pi_{\theta_{\text{old}}}$ rather than differentiated through a logarithm directly.

Because the from-scratch requirement of this project is discharged by REINFORCE and DQN, PPO is used via the reference Stable-Baselines3 implementation, with the same environment, data split, fee, budget and seeds as the scratch methods.

3.10 Algorithms

Algorithm 1 REINFORCE for the trading MDP (as implemented)

```

1: initialise policy network  $\pi_\theta$ 
2: for iteration = 1, 2, ... until step budget exhausted do
3:   sample a training month; reset environment ( $C_0$  cash, no position)
4:   for  $t = 0, \dots, T - 1$  do
5:     compute legal-action mask; set illegal logits to  $-\infty$ 
6:     sample  $a_t \sim \pi_\theta(\cdot | s_t)$ ; execute; store  $(\log \pi_\theta(a_t | s_t), r_t)$ 
7:   end for
8:   compute rewards-to-go  $G_t$ ; standardise within the episode
9:    $L(\theta) \leftarrow - \sum_t \log \pi_\theta(a_t | s_t) G_t$ 
10:  one Adam step on  $\nabla_\theta L$ 
11: end for
```

Algorithm 2 Deep Q-learning for the trading MDP (as implemented)

```

1: initialise  $Q_\phi$ ; target  $Q_{\phi^-} \leftarrow Q_\phi$ ; replay buffer  $\mathcal{D}$ 
2: for environment step = 1, 2, ... until step budget exhausted do
3:   with prob.  $\epsilon$ : random legal action; else  $a_t = \arg \max_{a \text{ legal}} Q_\phi(s_t, a)$ 
4:   execute  $a_t$ ; store  $(s_t, a_t, r_t, s_{t+1}, \text{mask}_{t+1}, d_t)$  in  $\mathcal{D}$ 
5:   sample minibatch; form masked targets  $y$  per (3.16); Adam step on  $L(\phi)$ 
6:   every  $K$  steps:  $\phi^- \leftarrow \phi$ 
7: end for
```

3.11 Evaluation protocol

Split. Chronological and three-way, on calendar boundaries: 60 training months (2018–2022), 12 validation months (2023) and 24 held-out test months (2024–2025). Every design and hyper-parameter decision is taken on the validation year; the test years are evaluated once. A two-way split would leave no way to demonstrate that the choice of learning rate, network width or step budget had not been influenced by the months used to report results.

Evaluation. Greedy, deterministic action selection with the same feasibility mask used in training. Every method sees identical episodes, identical initial cash ($C_0 = \$10,000$) and identical fees (5 basis points).

Reference strategies. Four, and the distinction between the two passive ones matters.

- **B0, do-nothing:** always Hold. Preserves cash exactly, so any method below it destroys value.
- **B1a, slice schedule:** buy one slice whenever a Buy is legal. This is the best a strategy restricted to the agent’s own action granularity can do, so comparing against it isolates timing skill from any advantage in action representation.
- **B1b, buy-and-hold:** fully invested at the first bar and held to forced liquidation. This is the passive benchmark an investor would actually use, and the harder of the two to beat in a market that drifts upward.
- **B2, random legal:** uniform over legal actions, from a single generator threaded through all test months, giving the chance floor.

Reporting both passive references prevents a familiar error. A schedule that buys a tenth of capital per step needs ten steps to become fully invested and so holds well under full exposure for much of a month; presenting it as buy-and-hold understates the benchmark and flatters whatever is compared against it.

Metrics. With monthly returns $R_i = \Delta W_i / C_0$ over the $M=24$ test months:

- **Mean and standard deviation of ΔW :** central tendency and month-to-month variability of wealth change, in dollars.
- **Trades per month:** a behavioural fingerprint, and with fees an economic cost the agent should manage.
- **Mean exposure:** the average fraction of capital at risk, without which return comparisons are not comparable.
- **Win rate against B1b:** fraction of months with strictly higher ΔW than buy-and-hold.
- **Monthly Sharpe ratio** [28]: $\bar{R} / \sigma_R$, return earned per unit of return volatility. Interpreted with care, because near-zero exposure shrinks the denominator faster than the numerator, so Sharpe is only meaningful between strategies of comparable exposure.
- **Intra-episode maximum drawdown:** the largest peak-to-trough fall *within* a month, reported as a mean and a worst case. This is the loss a trader actually lives through.

- **Drawdown of accumulated monthly profit:** the same measure applied to monthly profits accumulated on a constant stake. It describes month-to-month consistency and is reported separately, because it is a different quantity from the one above and not a conservative approximation of it.

Admissible configurations. State and reward cannot be crossed freely, because one combination is not an MDP. Of the four pairings of $\{S_3, S_4\}$ with the risk-neutral reward (3.4) and the risk-aware reward (3.3), three are admissible: S_3 with the risk-neutral reward is the baseline; S_4 with the risk-neutral reward is the control that isolates the cost of the extra feature; and S_4 with the risk-aware reward is the treatment. The fourth, S_3 with the risk-aware reward, is excluded by Section 3.3.2. A separate comparison varies the action model at the frozen state, so that state, reward and action representation are each varied one at a time.

Stopping rule for the ladder. Declared before any run: the ladder of Section 3.3.3 terminates at the first rung where the next rung changes mean ΔW , intra-episode maximum drawdown and trades per month by less than the spread observed across seeds $\{42, 43, 44\}$. Fixing the criterion in advance makes the stopping point a result rather than a consequence of the time available.

Standing diagnostics. (i) the accounting identity $\sum_t \Delta W_t = W_T - W_0$ in every episode, checked on the wealth change rather than the reward, since under (3.3) the two differ by the penalty; (ii) the illegal-action proposal rate; (iii) all learned results reported over seeds $\{42, 43, 44\}$.

Chapter summary (plain language). The trading problem is now fully specified. Each month the agent starts with \$10,000 in cash and sees nine numbers a human day-trader could also work out: how the price just moved, whether that move is part of a trend, how choppy the market is, how far price sits from its recent average, how wide the day's range was, how much cash it holds, what its position is worth, whether that position is up or down, and how many days remain. It may hold, buy a \$1,000 slice, or sell one; it pays a small fee on every trade; and it is scored by the change in its total wealth.

Those nine numbers were not chosen by taste. Each one answers a situation the previous version of the state could not tell apart, and one candidate was deliberately left out because arithmetic already available to the network could reconstruct it. A tenth number, how far wealth has fallen from its high point, is held back for a separate experiment, because it is only needed once the score itself starts penalising risk.

Two learning rules were then derived from that score: REINFORCE, which strengthens whatever decisions preceded good months, and where the logarithm in the formula is forced by the calculus rather than chosen; and Deep Q-learning, which learns a running price list for the three choices. PPO, a modern stabilised variant, joins as a third contestant. The next chapter turns these equations into tested code.

Chapter 4

System Design & Implementation

This chapter describes how the mathematical formulation of Chapter 3 is realised in code. The guiding requirement, set explicitly by the supervision process, is that every implemented line be traceable to a stated equation: the policy-gradient loss in the code *is* Equation 3.14; the temporal-difference loss *is* Equation 3.16. Both learning algorithms are implemented from scratch in PyTorch [32] with the Adam optimiser [33]; environments follow the Gymnasium API [34, 35]. No reinforcement-learning library performs any part of the final-model training.

4.1 Architecture overview

The system separates six concerns, each in its own module:

1. **Feature registry** (`features.py`): the single definition of every feature, its canonical ordering, the rolling window lengths, and the ladder of Section 3.3.3 expressed as named subsets. Nothing else in the codebase decides what a state contains.
2. **Data pipeline** (`data.py`): downloads OHLCV bars, computes market features on the continuous series, cuts monthly episodes and applies the three-way chronological split; run once and cached to disk.
3. **Environment** (`env.py`): a Gymnasium `gym.Env` implementing the MDP — state assembly, trade execution, fees, masking, termination and the admissibility guard.
4. **Agents** (`agents.py`): the two from-scratch learners, exposing a common `policy(obs, mask) → action` interface after training, plus a thin adapter that gives the third-party PPO the same interface.
5. **Reference strategies and rollout** (`baselines.py`, `rollout.py`): the four benchmarks and the single episode-runner and metric summariser they share with the learned agents.
6. **Experiment harness** (`harness.py`, driving `run_ladder.py` and `run_final.py`): trains every method under an identical budget and writes a JSON record of the configuration, per-month results, diagnostics and provenance.

Two properties of this arrangement do methodological work rather than merely organising code. Because baselines, learned agents and diagnostics all flow through the same `run_episode` function, no method can be advantaged by a differing evaluation path. And because both experiment scripts obtain their environments from the same `harness.Config` object, any difference between two reported rows must appear in the configuration dictionary stored alongside them; there is no route by which two rows could differ in a way the results file does not record. Figure 4.1 summarises the flow.

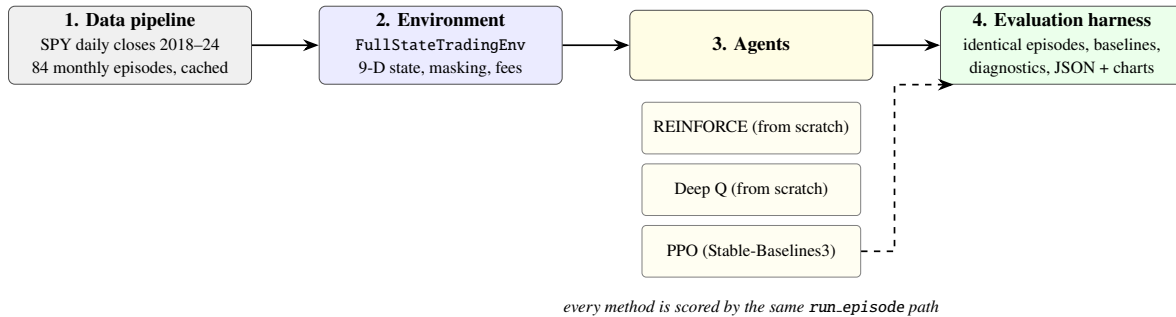


Figure 4.1: System architecture: four decoupled modules. The evaluation harness scores every learned agent and every baseline through one shared episode-runner, so no method can gain from a differing evaluation path.

4.2 Data pipeline

Daily auto-adjusted OHLCV bars for SPY (the S&P 500 ETF) covering January 2018 to December 2025 are retrieved via the `yfinance` API. SPY is chosen deliberately: it is among the most liquid instruments in the world, so the frictionless-execution assumption of the environment (trades fill at the closing price) is least violated, and as a broad index fund it carries no single-company idiosyncrasies. The full open, high, low, close and volume record is retained rather than the closing price alone, because the daily high and low are what make range-based risk representable at all (Section 3.3.4).

Features are computed before episodes are cut. The pipeline computes every market feature on the continuous price series and only then slices it into months. This ordering is the whole point of the module. Computing a 20-day trend reference inside a monthly episode would leave the first day of every month with no history to average, so the agent would be told the market has no trend and no volatility at precisely the moment it chooses its opening position. Momentum does not reset because a new month has begun.

Making that ordering work requires one further step. A rolling window needs history *before* the first day it is reported for, so the pipeline fetches a warm-up lead from September 2017, computes features across the join, and only then discards the lead. The longest window in use is 20 bars, so `WARMUP_BARS` is 21 and the pipeline refuses to proceed if fewer lead bars are available than that. It also asserts that no missing value survives into any retained episode, which is the check that would catch a silently shortened lead.

Episodes and the split. Days are grouped into calendar months and months with fewer than 15 trading days are dropped, giving 96 monthly episodes. These are split chronologically and on calendar boundaries

(Figure 4.2): 60 training months (2018–2022), 12 validation months (2023) and 24 test months (2024–2025). The split is never shuffled, so the test period lies strictly in the future of every training sample, which is the temporal analogue of avoiding data leakage (Section 2.6).

The validation year exists to carry decisions. The saturation study that fixes where the state ladder stops, and the choice of the penalty weight λ , are both settled on 2023 and never on the test years. Under the two-way split used earlier in the project there was no way to make that separation, and therefore no way to demonstrate that a design choice had not been influenced by the months used to report the final numbers. Episode arrays and metadata are cached as `.npz` and `.json`, so every experiment is reproducible offline and no run can silently train on differently-prepared data.

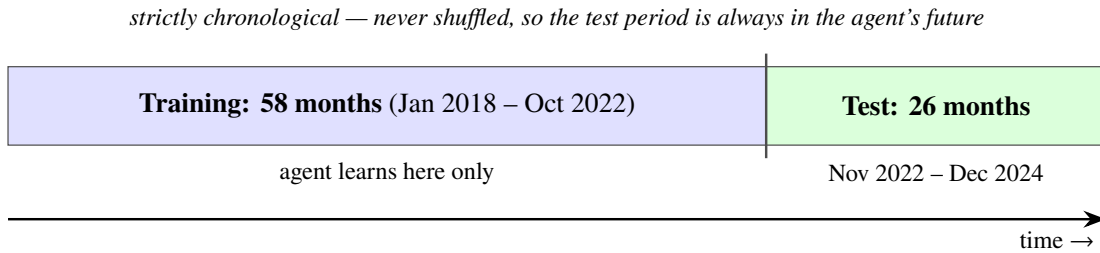


Figure 4.2: Chronological three-way split of the 96 monthly episodes: 60 training months, 12 validation months and 24 held-out test months.

The eight-year window is not homogeneous, which is deliberate. Training data includes the 2018 volatility spikes, the 2020 COVID crash and rebound, and the 2022 bear market; validation covers the 2023 recovery; and the held-out test window covers 2024 and 2025. An agent trained on this window has met falling markets, so it cannot excuse its test behaviour by never having seen one.

4.3 The trading environment

`TradingEnv` subclasses `gym.Env` with `action_space = Discrete(3)`. Internally it tracks cash C_t , fractional units h_t , the cost basis of the open position (average-cost accounting) and the running peak of wealth, from which the account features are computed on demand.

The environment receives features rather than computing them. The constructor takes a precomputed market matrix aligned to its price series and validates its width against the feature registry. This is a structural guarantee rather than a convenience: an environment that never computes a rolling window cannot truncate one at an episode boundary. The defect described in Section 4.7 is not fixed here so much as made unrepresentable.

The observed feature set is selectable. The constructor takes a list of feature names and emits observations in the canonical order of `features.py` filtered by that list, so `observation_space` is $\mathbb{R}^9$ for the state of Table 3.4 and $\mathbb{R}^{10}$ for the risk-aware extension. Every rung of the ladder therefore runs through one code path, which means a measured difference between two rungs cannot be an implementation difference between two environments.

Admissibility is enforced at construction. If `risk_lambda` is positive and `drawdown` is not among the requested features, the constructor raises. The argument of Section 3.3.5 is therefore checked by the program rather than asserted in prose: the inadmissible pairing of S_3 with a risk-aware reward cannot be instantiated, whether by a script, by a command-line argument or by mistake. `run_final.py` attempts the construction deliberately and records that it was refused, so the guard is evidenced in the results file.

Trade execution and the action model. Under the default slice model a Buy spends $\min(C_t, \Delta)$ dollars with $\Delta = 0.1 C_0$, acquiring $x/(P_t(1+c))$ units, and a Sell liquidates $\min(h_t P_t, \Delta)$ of position value, crediting $u P_t(1-c)$. Under the whole-share model a Buy spends one share price and a Sell releases one unit. Both are implemented, selected by a constructor argument, because Section 3.4 treats action representation as an experimental axis in its own right. Residual position value below the \$1 minimum trade size is flushed to cash, so that “flat” means flat and dust cannot accumulate outside the mask logic.

Masking and safety. `action_masks()` returns the legal action set each step. If a caller nevertheless submits an illegal action, the environment executes Hold instead and flags the event in `info`; the harness accumulates this as the illegal-action diagnostic.

Reward and its bookkeeping. `info` carries the pure wealth change `dw` and the risk penalty separately from the returned reward. This separation is what allows the accounting identity to be checked under a shaped reward, and it lets a risk-trained agent be *evaluated* on wealth change so that it is compared with the other agents on the same quantity it will ultimately be judged by.

Termination. At the final bar the environment force-liquidates any open position at the closing price with the fee applied, implementing the mandatory end-of-episode liquidation of Section 3.4.

4.4 From-scratch REINFORCE

The policy network is a two-hidden-layer MLP ($9 \rightarrow 128 \rightarrow 128 \rightarrow 3$, tanh activations). Sampling uses a masked categorical distribution: illegal logits are set to -10^9 before the softmax, so probability mass is renormalised over legal actions only — the network is never trained on, and can never propose, an illegal trade.

One gradient update is performed per episode. The core of the training loop is four lines, in exact correspondence with the derivation of Section 3.7:

```
# rewards-to-go, standardised within the episode
G[t] = r[t] + gamma * G[t+1]
G = (G - G.mean()) / (G.std() + 1e-8)
loss = -(log_probs * G).sum()      # L(theta) = -sum_t log pi(a|s) * G_t
loss.backward(); optimizer.step() # gradient ascent on J
```

The returns tensor carries no gradient (it is data, not a function of θ in the surrogate loss), matching Step 6 of the derivation.

4.5 From-scratch Deep Q-learning

The Q-network is an MLP of the same capacity ($9 \rightarrow 128 \rightarrow 128 \rightarrow 3$, ReLU). The implementation contains the three standard DQN components, each present for a reason stated in Section 3.8:

- **Replay buffer** (50k transitions, uniform sampling, minibatch 64): breaks the temporal correlation of consecutive market steps.
- **Target network** (synchronised every 500 steps): stabilises the bootstrapped TD target.
- **Masked ε -greedy** ($\varepsilon : 1.0 \rightarrow 0.05$ over the first half of training): exploration restricted to legal actions; the max in the TD target is likewise taken over the *stored* legal-action mask of the next state.

Storing the next-state mask in the buffer is a subtle but necessary detail: without it, the target could bootstrap from an action that is illegal in s_{t+1} (e.g. Sell while flat), silently overestimating value.

The core update, in exact correspondence with Equation (3.16):

```
q_next = target_net(s_next)                # Q_phi-minus(s', .)
q_next[~mask_next] = -1e9                  # max over LEGAL actions only
y = r + gamma * (1 - done) * q_next.max(dim=1).values
loss = mse(qnet(s).gather(1, a), y.detach()) # TD loss on Bellman targets
loss.backward(); optimizer.step()
```

The `y.detach()` matters: the target is data, not a function to differentiate through — gradients flow only into $Q_\phi(s, a)$, exactly as the semi-gradient TD rule prescribes [12].

4.6 PPO ablation row

The third method, PPO, uses a reference implementation rather than a from-scratch one, because its role is to answer whether a modern stabilised policy-gradient method behaves differently on this problem, and a hand-written approximation of PPO would confound that question with the quality of the reimplement.

Masking, and why the library choice matters. Plain PPO from Stable-Baselines3 [11] has no native action masking. Using it would leave the policy free to place probability mass on infeasible trades and rely on the environment’s forced-Hold override to absorb them, which is a materially different learning problem from the one REINFORCE and Deep Q-learning solve: their distributions are renormalised over legal actions before any action is sampled. Comparing a masked learner with an unmasked one and attributing the difference to the algorithm would be unsound. PPO is therefore taken from `sb3-contrib` in its `MaskablePPO` form [11], which applies the same feasibility constraint through the same `action_masks()` call, so all three methods face an identical action space at every step.

Matching the episode distribution and the seed. SB3 treats an environment as one long-running task, whereas the trading problem is a distribution over months. A thin wrapper (`MonthSampler`) draws a new training month on every `reset()` from a generator seeded by the run seed. The seeding is the point. In the earlier implementation this sampler was seeded to a constant, so changing the run seed altered only the

network initialisation and left the sequence of months identical; PPO's reported seed-to-seed spread then measured less variation than the other two methods' did, and the three were not comparable on stability. The wrapper now takes the run seed, so every method's spread reflects the same two sources of variation: initialisation and the order in which months are seen.

4.7 What verification found

Verification is reported here as a result rather than a formality, because it changed the experiment. An audit of the earlier implementation, carried out before the final runs, found twelve defects, of which three were severe enough to invalidate the numbers the earlier version produced. Two were quantified directly on the real data by a dedicated script.

Rolling features were truncated at episode boundaries. The earlier environment computed momentum and volatility inside each episode from that episode's prices alone. Every month therefore opened with a momentum of exactly zero and a volatility of exactly zero, values that were not measurements but artefacts of where the calendar had been cut. On the real series, 43% of each episode's steps carried a window that was partially or wholly unfilled. The agent was being trained to open positions on the basis of a feature that was, at that moment, guaranteed to be blank.

The buy-and-hold benchmark was not buy-and-hold. The earlier baseline bought one slice whenever a Buy was legal, inside an environment whose slice was a tenth of capital. Such a policy needs ten steps to become fully invested and so averages roughly 0.69 exposure across a month rather than 1.0. It is a ten-day dollar-cost average wearing the name of buy-and-hold, and because the asset drifts upward it is systematically weaker than the strategy it was labelled as. Measured on the test months, true buy-and-hold earns 21.6% more than the policy previously reported under that name. Every agent compared against it was flattered by the difference. Both are now reported, under names that say what they are, and buy-and-hold is pinned to the slice action model so that it means the same thing even in configurations where the agent trades whole shares.

The risk-aware reward was not measurable from the state. The earlier experiment penalised drawdown while observing a state that did not contain it, which is the violation Section 3.3.2 describes. The result was not an MDP, so the derivations of Chapter 3 did not apply to it and its outcome could not be interpreted. This is now prevented by construction rather than corrected by convention.

The remaining nine defects were smaller but pointed the same way: closing prices retained while the rest of the OHLCV record was discarded; a two-way split with no validation set; data ending a year earlier than available; the PPO seeding and masking asymmetries described above; drawdown measured only across months and never within one; a feature that carried no information; and an accounting check applied to the shaped reward, where it must fail for a correct environment, rather than to the wealth change. Each is recorded with its fix, and the audit is what motivated rebuilding the experiment package rather than patching it.

The general lesson is worth stating, because it is the one an examiner is entitled to ask about. None of these defects announced itself. Every one produced plausible numbers, and the two quantified above both biased results in the flattering direction. They were found by asking what each component was supposed

to guarantee and then testing that specific claim, which is why the following section describes the tests in terms of the properties they defend rather than the functions they call.

4.8 Verification and testing

Two layers of verification precede any market experiment.

Unit tests. Twenty-one tests (`test_v2.py`) are organised around the properties they defend rather than the functions they exercise. All pass.

Feature contracts. No feature may use data from after the bar it is reported for; the test truncates the series at t , recomputes, and requires the values at t to be unchanged. Rolling features must be continuous across a month boundary; the test computes them on a continuous series and on the concatenation of its monthly pieces and requires the first to differ from the second, which is the presence of the defect being asserted as absent from the pipeline.

MDP contracts. The accounting identity $\sum_t \Delta W_t = W_T - W_0$ must hold under random legal play, checked against the wealth change rather than the reward. With $\lambda = 0$ the reward must equal the wealth change exactly. With $\lambda > 0$ and drawdown absent, construction must raise.

Feature relationships. The episode’s running profit must be exactly recoverable from cash and exposure, which is the arithmetic that justifies excluding it. Drawdown must *not* be recoverable: the test exhibits two states that agree on every other feature and differ in drawdown, so its inclusion is shown to be necessary rather than argued to be.

Behavioural contracts. Sell is masked while flat and Buy once cash is exhausted; the illegal-action override engages and is reported; forced liquidation leaves zero units; the clock runs from 0 to 1; the two action models differ in the direction claimed; each reference strategy attains the exposure its description promises; and the data pipeline’s three splits are non-empty, disjoint and chronologically ordered.

Algorithm validation on known-outcome tasks (Phase 0). Before touching market data, the same algorithm implementations were validated on games where success is unambiguous: CartPole-v1, Flappy Bird and LunarLander-v3. Every learner clears its random floor (e.g. CartPole: random 28.8 vs. REINFORCE 292 and DQN 500/500). A learning algorithm that cannot solve a game with a known answer cannot be trusted on markets, where the correct answer is unknown — this ordering was a fixed methodological requirement of the project.

4.9 Experimental configuration

4.10 Reproducibility

Every claim in Chapter 5 is regenerable from committed code and cached data. Specifically: (i) all random number generators, in Python, NumPy and PyTorch, are seeded, and results are reported over three seeds rather than a favourable one; (ii) the data pipeline caches its output, so experiments never silently retrain on differently-prepared data; (iii) each experiment writes a JSON record containing the configuration, per-month results, training traces, diagnostics and provenance, the last including the git

Table 4.1: Hyperparameters and budgets for the final experiments. d is the observation width, 9 for the state of Table 3.4 and 10 for the risk-aware extension.

Setting	REINFORCE	Deep Q	MaskablePPO
Network	MLP $d-128-128-3$, tanh	MLP $d-128-128-3$, ReLU	MLP 128-128
Optimiser / lr	Adam, 10^{-3}	Adam, 10^{-3}	Adam, 3×10^{-4}
Discount γ	0.99	0.99	0.99
Update cadence	once per episode	every step after warm-up	every 2,048 steps
Replay / batch	—	50k buffer, batch 64	batch 64, 10 epochs
Target sync	—	every 500 steps	—
Exploration	sampling from π_θ	ϵ : 1.0 $\rightarrow$ 0.05 (first 50%)	entropy bonus
Masking	renormalised softmax	masked max, mask stored	<code>action_masks()</code>
Budget	80,000 env steps	80,000 env steps	80,000 env steps
Seeds	{42, 43, 44}	{42, 43, 44}	{42, 43, 44}
Environment	$C_0 = \$10k$; fee 5 bps; slice $\Delta = 0.1 C_0$; windows $k=5, m=20, n=14$		
Selection	ladder stopping point and λ chosen on the 2023 validation months only		

commit and whether the working tree was clean when the run started, so a result can always be traced to the exact code that produced it; and (iv) the whole suite runs on CPU in about an hour, with no result depending on specialised hardware. That last property is deliberate: every number in this dissertation can be regenerated from scratch during a viva demonstration.

Two selection decisions are also recorded in the results files rather than described only in prose. The ladder’s stopping rule and the rule for choosing λ are both evaluated in code, each carries a flag recording that it was declared before the runs, and each stores the comparisons it made. A reader can therefore check that the stopping point follows from the stated criterion instead of taking that on trust.

Chapter summary (plain language). The mathematics of Chapter 3 now exists as a handful of small, separately testable programs: one that fetches and organises eight years of real market data; one that simulates the trading month, charging fees and enforcing the rules; two learners built from the ground up, each of whose central lines of code is a direct transcription of a derived equation; and a harness that runs every method and every benchmark through exactly the same evaluation path.

Two design decisions in that code are doing more work than they appear to. The simulator is handed its market indicators rather than computing them, which makes it structurally incapable of the boundary defect described above. And it refuses to be built at all if the score it is asked to use depends on something the agent cannot see, so the consistency argument of Chapter 3 is enforced by the program instead of promised in prose.

Before being allowed near market data, both learners had to prove themselves on video games with known answers, and the simulator had to pass twenty-one tests, including that its per-step wealth changes always sum exactly to the month’s true profit. Those checks were not a formality: they turned up twelve real defects in the earlier version of this work, two of which had been quietly making the results look better than they were. The next chapter reports what happened once they were fixed.

Chapter 5

Empirical Results & Analysis

This chapter reports every experiment in the order the methodology was built: algorithm validation on games with known outcomes (Section 5.2); a pilot study with a deliberately minimal state whose failure motivates the final design (Section 5.3); and the main comparison of from-scratch REINFORCE and Deep Q-learning on the complete 9-feature state (Section 5.4), followed by the fee-sensitivity study (Section 5.5), a preliminary risk-aware-reward experiment (Section 5.6) and the consolidated analysis (Section 5.7). All numbers are produced by committed scripts from cached data; none are illustrative placeholders. Complete per-month results for every method appear in Appendix B.

For a reader keeping only one question in mind, let it be this one: *did any learned agent do better than simply buying at the start of the month and holding to the end?* Everything in this chapter is organised to answer that question fairly — and the honest answer turns out to be more interesting than a yes or a no.

5.1 Experimental setup

Data and split. SPY daily closes 2018–2024; one calendar month of daily bars per episode; chronological split of 58 training / 26 held-out test months (test = November 2022 – December 2024).

Protocol. Initial cash $C_0 = \$10,000$; proportional fee $c = 5$ bps; trade slice $\Delta = 0.1 C_0$; training budget 80,000 environment steps per method per seed; seeds {42, 43, 44}; greedy masked evaluation. Metrics and baselines as defined in Section 3.11.

5.2 Phase 0: algorithm validation on games

Before any market experiment, the learning implementations were validated on tasks with unambiguous success criteria (Table 5.1). Every learner clears its random floor. Two observations carry forward. First, from-scratch REINFORCE learns but plateaus below the more stabilised methods on CartPole (292 vs. 500) — the textbook variance limitation of Monte-Carlo policy gradients, visible in our own runs. Second, tabular Q-learning solves CartPole *only after* hand-discretizing the 4-dimensional continuous state; the 9-dimensional continuous trading state makes any such table infeasible, which is precisely why the value-based track uses a deep Q-network (Section 3.8).

Table 5.1: Phase 0 mean evaluation return (30 greedy episodes, seed 42). CartPole solved ≈ 475 ; LunarLander solved ≈ 200 .

Algorithm	CartPole-v1	FlappyBird-v0	LunarLander-v3
Random	28.8	-7.4	-183.4
Tabular Q-learning	500.0	—	—
DQN	500.0	7.0	177.6
REINFORCE (scratch)	292.2	7.1	15.3
A2C	500.0	4.6	-41.2
PPO	500.0	12.6	176.4

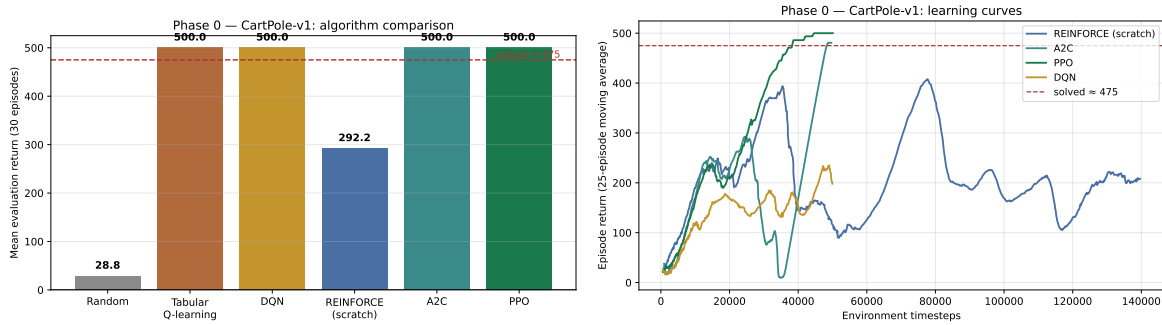


Figure 5.1: CartPole-v1: final comparison (left) and learning curves (right, 25-episode moving average). Scratch REINFORCE learns but plateaus below the stabilised methods — the variance limitation predicted by the theory of Section 3.7.

5.3 Pilot study: the minimal state collapses to buy-and-hold

The pilot used the minimal 3-feature state $s = [\Delta P, C, n]$ with whole-share actions and the same wealth-change reward, split, fee and budget. Three policy-gradient optimisers — from-scratch REINFORCE, A2C and PPO — were trained on the identical MDP.

Table 5.2: Pilot (minimal 3-D state, whole shares): all learned policies execute an action sequence identical to fully-invested buy-and-hold on every one of the 26 test months.

Method	Mean ΔW (\$)	Trades/mo	Months identical to buy-max
Buy-max-hold (baseline)	76.91	19.1	26/26 (by definition)
REINFORCE (scratch)	76.91	19.1	26/26
A2C	76.91	19.1	26/26
PPO	76.91	19.1	26/26

Two lessons shaped the final design. *Baseline design is itself a result*: against a naive one-share buy-and-hold baseline (mean $\Delta W = \$7.97$), any of these agents would have appeared to win 10 $\times$; only the fair fully-invested baseline reveals that they had simply rediscovered buy-and-hold. *The collapse is a property of the objective and the state, not of any single optimiser*: three different policy-gradient methods found the same optimum. The final model therefore enriches the state (Section 3.3) and adds a value-based learner as a structurally different second method.

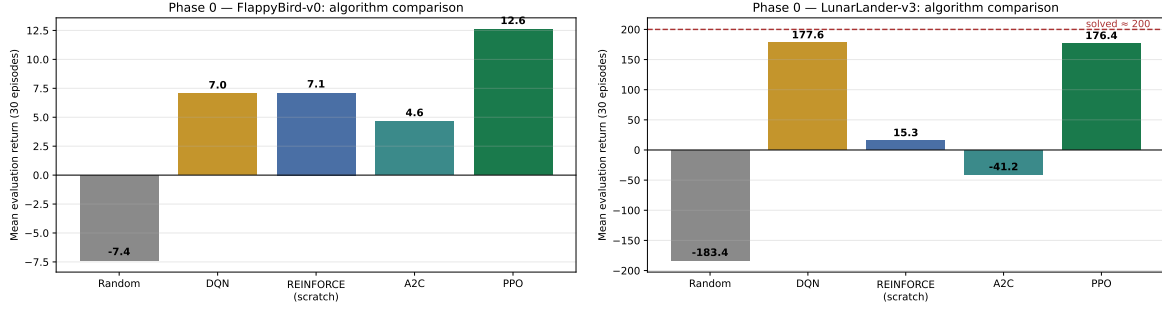


Figure 5.2: Flappy Bird (left) and LunarLander-v3 (right): every learner clears its random floor under a fixed budget.

5.4 Main results: the algorithm ablation on the complete state

Table 5.3 and Figure 5.3 report the full ablation on the 26 held-out months: the two priority methods (from-scratch REINFORCE and Deep Q) and the PPO stretch row, against all baselines, under a matched 80k-step budget (primary seed 42; seed spread in Table 5.4).

Table 5.3: Algorithm ablation on 26 held-out SPY test months (seed 42, 80k steps each). ΔW in dollars. Sharpe on monthly returns; MDD on the concatenated test equity curve. “Id. B1b” counts months whose executed action sequence is identical to buy-and-hold. Accounting identity held to $< 10^{-6}$ for every method on every month; the illegal-action rate was zero for all evaluated policies.

Method	Mean ΔW	Std	Trades/mo	Win vs B1b	Sharpe	MDD	Id. B1b
B0 do-nothing	0.00	0.00	0.0	31%*	—	0.0%	0/26
B2 random (masked)	37.97	80.43	14.2	31%	0.47 [†]	0.9%	0/26
B1b buy-and-hold	125.19	309.87	11.0	—	0.40	6.0%	26/26
REINFORCE (scratch)	125.19	309.87	11.0	0%	0.40	6.0%	26/26
Deep Q (scratch)	123.23	273.97	13.2	35%	0.45	4.8%	0/26
PPO (SB3)	55.67	110.45	4.0	31%	0.50	1.9%	0/26

*B0 “wins” only in falling months, by not being invested. [†]Sharpe ratios of low-exposure strategies (B2, PPO) are inflated by construction: near-zero exposure gives tiny variance. Sharpe comparisons are meaningful only between strategies of comparable exposure.

REINFORCE rediscovers buy-and-hold — exactly. The REINFORCE policy’s executed trades match fully-invested buy-and-hold to the cent on all 26 test months (win rate vs. B1b = 0 because the wealth paths are identical; Figure 5.4, dashed line). Even with entry, clock, trend and risk features available, the Monte-Carlo policy gradient converges to “invest everything immediately and hold.” This is not an implementation failure; it is the risk-neutral optimum. Under $r_t = \Delta W_t$, expected return is maximised by maximum exposure whenever expected drift is positive — and the 2018–2022 training window drifts upward on average. The 9-feature state changes what the policy *could* express, but not what the objective *rewards*.

Deep Q deviates — modestly, and in the direction of risk control. The DQN policy executes a different action sequence in every month, wins against buy-and-hold in 9/26 months (predominantly falling ones), and achieves lower monthly standard deviation (274 vs. 310), lower maximum drawdown (4.8% vs. 6.0%) and higher monthly Sharpe (0.45 vs. 0.40), at a cost of \$1.96/month of mean return. Figure 5.5

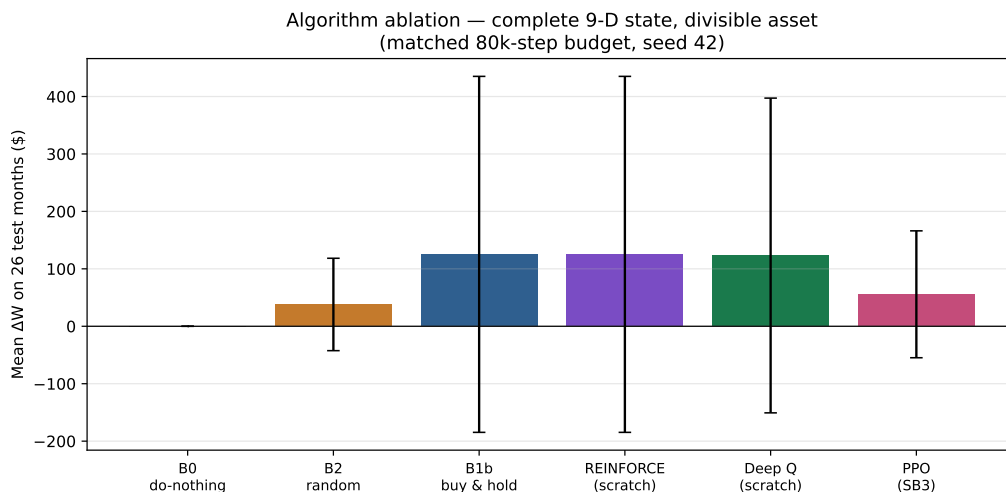


Figure 5.3: Mean test-month wealth change by method (error bars: one standard deviation across months).

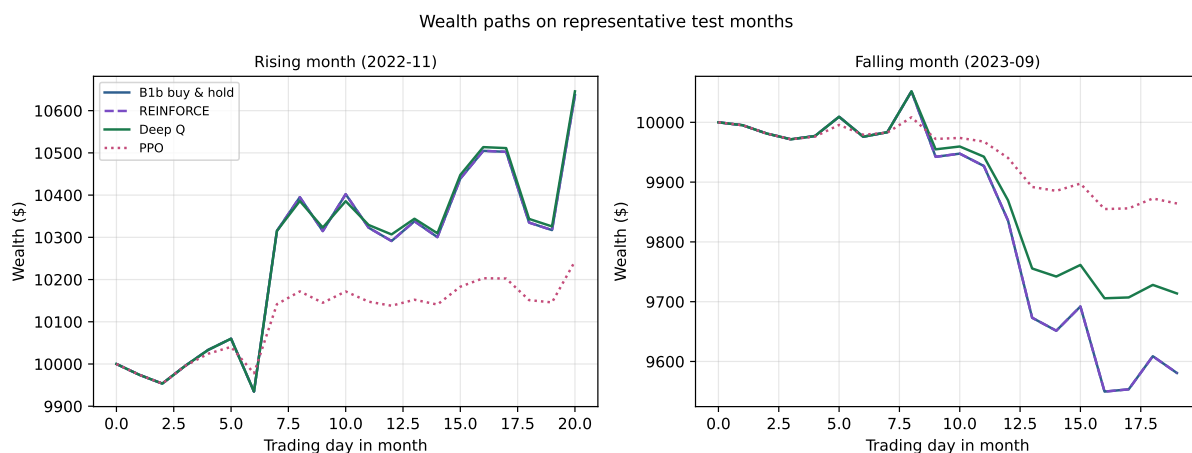


Figure 5.4: Wealth paths on the strongest rising test month (2022-11, left) and the weakest falling month (2023-09, right). REINFORCE overlays buy-and-hold exactly. Deep Q tracks it in the rising month but exits progressively during the decline. PPO (seed 42) stays lightly invested in both.

shows the mechanism on the worst test month: both agents ramp to near-full investment, but as the decline develops Deep Q sells down to $\approx 40\%$ exposure while buy-and-hold stays fully invested. Mechanistically, bootstrapped one-step targets make DQN sensitive to short-horizon state distinctions (local volatility, negative momentum) that the Monte-Carlo return signal averages away; the learned behaviour is “fully invested by default, trim around turbulence.”

PPO is bimodal across seeds. Because a two-mode claim needs more than three samples, PPO was run on five seeds. On seeds 43, 44 and 45 it converged to *exact* buy-and-hold (26/26 identical months, $\Delta W = \$125.19$); on seeds 42 and 46 it converged to a cautious low-exposure policy ($\Delta W \approx \$56$, ≈ 4 trades/month, $MDD \approx 1.9\%$) — each mode replicated at least twice, with nothing in between. Both corners are rational under the risk-neutral objective; which one PPO reaches depends on its early rollouts, because ratio clipping deliberately limits how far each update can move the policy — slow movement stabilises training but can freeze the policy near wherever the early data pointed. This seed sensitivity is itself an ablation finding: the two scratch methods were directionally consistent across all seeds; the

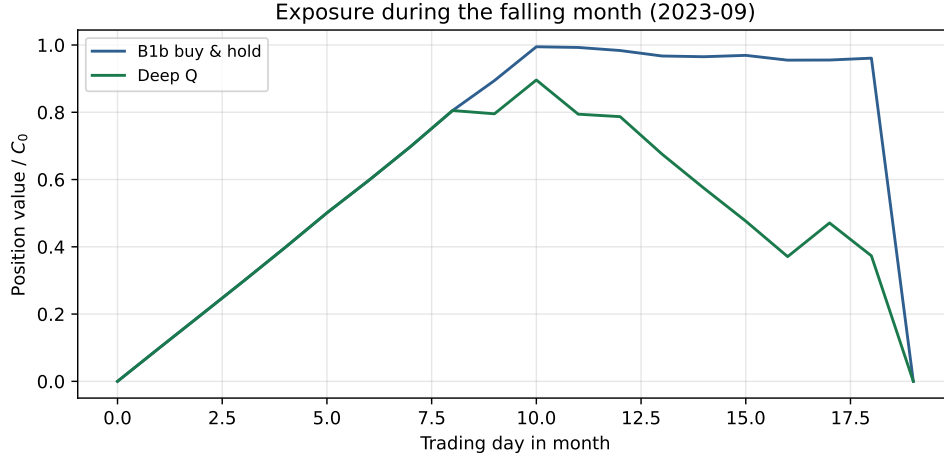


Figure 5.5: Exposure (position value as a fraction of C_0) during the falling month 2023-09. Both agents ramp to near-full investment; as the decline develops, Deep Q trims exposure from ≈ 0.9 to ≈ 0.4 while buy-and-hold remains pinned at ≈ 1.0 until forced terminal liquidation.

clipped method was not.

Table 5.4: Seed spread: mean ΔW in dollars (months identical to buy-and-hold in parentheses). PPO was additionally run on seeds 45–46 to test the bimodality claim.

Method	Seed 42	Seed 43	Seed 44	Seed 45	Seed 46
REINFORCE	125.19 (26/26)	119.96 (0/26)	118.52 (0/26)	—	—
Deep Q	123.23 (0/26)	98.89 (0/26)	118.48 (7/26)	—	—
PPO	55.67 (0/26)	125.19 (26/26)	125.19 (26/26)	125.19 (26/26)	56.48 (0/26)

Across seeds, REINFORCE sits at or within a few dollars of the buy-and-hold corner every time (its seed-43/44 variants differ only by a handful of early-month trades); Deep Q consistently lands nearby with more trading and equal-or-better risk metrics; PPO oscillates between the two rational corners.

Training dynamics. Figure 5.6 shows episode returns during training for the scratch methods. Unlike the Phase-0 games, the curves do not “rise and saturate”: each training episode is a randomly drawn month whose achievable return is dominated by that month’s drift, so the reward signal is noisy by construction. This is the practical face of the high-variance problem analysed in Section 3.7, and a key difference between trading and game benchmarks: the environment’s randomness overwhelms the visible learning signal, and evaluation on held-out data — not the training curve — is the only trustworthy progress measure.

5.5 Fee sensitivity

To test whether the learned behaviours respond to economic friction, REINFORCE and Deep Q were retrained (seed 42) at three fee levels and evaluated at the same fee (Table 5.5, Figure 5.7).

Two observations. First, REINFORCE tracks buy-and-hold identically at every fee level — consistent with its collapse: a policy that trades exactly like buy-and-hold pays exactly buy-and-hold’s fees. Second, Deep Q *adapts its trading intensity to the fee*: 19.8 trades/month when trading is free, 13.2 at 5 bps, and only 6.1 at 20 bps. The direction of the adaptation is economically correct (the fee enters the reward, and

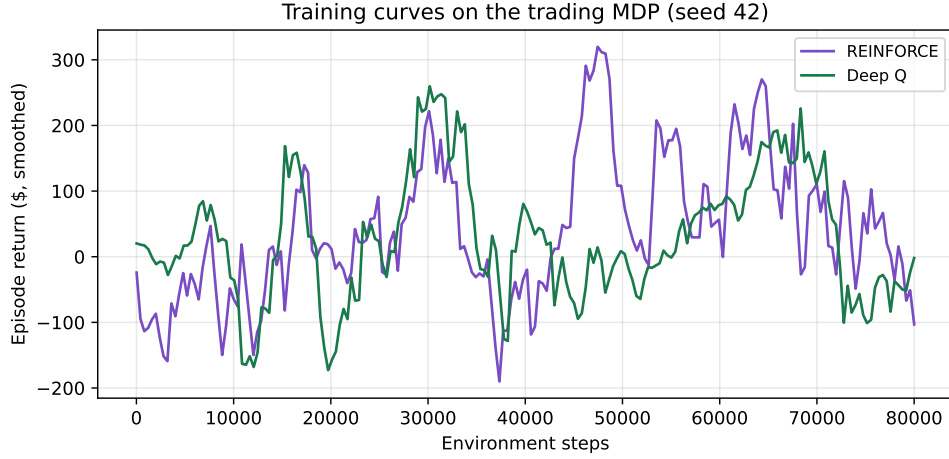


Figure 5.6: Training-episode returns (smoothed) for the scratch methods. Month-to-month drift dominates the visible signal; learning progress is measured on held-out months, not on this curve.

Table 5.5: Fee sensitivity (seed 42, retrained and evaluated per fee level). Mean ΔW in dollars; trades per month in parentheses.

Method	0 bps	5 bps	20 bps
B1b buy-and-hold	135.32 (11.0)	125.19 (11.0)	94.86 (11.0)
REINFORCE	135.32 (11.0)	125.19 (11.0)	94.86 (11.0)
Deep Q	111.09 (19.8)	123.23 (13.2)	65.90 (6.1)

the agent learns to economise on it), although at 20 bps the residual trading still costs more than it saves relative to buy-and-hold — risk trimming is not free, and a risk-neutral objective gives the agent no credit for the variance it removes.

5.6 Preliminary evidence: a risk-aware reward

The analysis so far repeatedly traces behaviour back to the risk-neutral reward. As a first, deliberately exploratory test of that attribution, both scratch methods were retrained (seed 42, all else identical) on a drawdown-penalised reward,

$$r'_t = \Delta W_t - \lambda \max(0, D_t - D_{t-1}), \quad D_t = \max_{u \leq t} W_u - W_t, \quad \lambda = 0.25,$$

which charges the agent a quarter of every new dollar of drawdown from its peak wealth. Crucially, evaluation still uses the *true* wealth metrics, so Table 5.6 is directly comparable to Table 5.3.

Table 5.6: Risk-aware preliminary (seed 42, $\lambda = 0.25$; trained on the shaped reward, evaluated on true wealth metrics).

Method	Mean ΔW	Std	Trades/mo	Sharpe	MDD
B1b buy-and-hold (reference)	125.19	309.87	11.0	0.40	6.0%
REINFORCE (risk-aware)	0.00	0.00	0.0	—	0.0%
Deep Q (risk-aware)	49.48	73.22	10.1	0.68	1.4%

The outcome sharpens the dissertation’s central claim in an unexpected way. REINFORCE does not

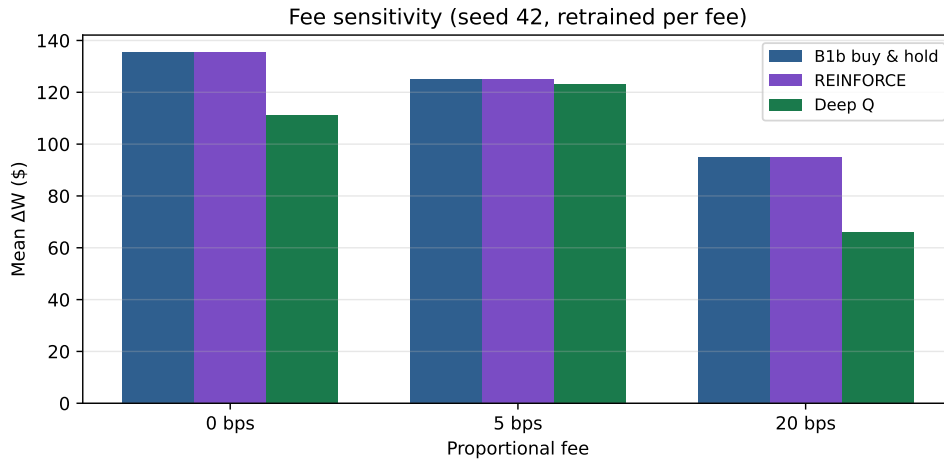


Figure 5.7: Mean ΔW under increasing proportional fees.

become a risk manager — it *jumps to the opposite corner*, converging to total abstention (zero trades, zero wealth change): under its episodic Monte-Carlo signal, once drawdowns are taxed at $\lambda = 0.25$ the all-cash policy dominates, and the corner-seeking character of the optimiser is unchanged — only the corner moves. Deep Q, by contrast, delivers precisely the trade the shaped reward requests: maximum drawdown collapses from 6.0% to 1.4% and the monthly Sharpe ratio rises from 0.40 to 0.68, in exchange for roughly 60% of the mean return. One seed and one λ make this preliminary evidence, not a study — but the direction confirms both the reward-attribution argument and the family-level behavioural difference, and defines the obvious next experiment (a λ sweep, Chapter 6).

5.7 Analysis and key findings

1. The reward, not the state (or the algorithm), sets the incentive. The pilot and the final model bracket this cleanly. With a minimal state, every optimiser collapsed to buy-and-hold. With the complete state, the Monte-Carlo policy gradient *still* chose buy-and-hold — because under a risk-neutral wealth-change reward in a rising regime, buy-and-hold is (near-)optimal, and REINFORCE optimises exactly that objective. Beating buy-and-hold on raw return in this setup would be evidence of a bug or of leakage, not of skill.

2. Algorithm families respond differently to the same objective. Under a matched budget, data split, fee and seed: REINFORCE’s episodic Monte-Carlo signal drives it to the exposure corner; Deep Q’s per-step bootstrapping yields a policy that reacts to local state, buys measurable risk reduction at negligible return cost, and adapts its trading intensity to transaction fees; PPO’s clipped updates make its destination seed-dependent, landing on either rational corner. This family-level behavioural characterisation is the central empirical finding of the dissertation.

3. Honest baselines remain the load-bearing methodology. Every headline in this chapter depends on the fully-invested baseline being present. The same table with only a one-share or do-nothing baseline would report a spurious 10× victory. Equally, the Sharpe column shows how a naive risk-adjusted comparison can mislead: the random and PPO (seed 42) rows post the highest Sharpe ratios simply because low exposure shrinks variance faster than it shrinks mean.

4. What a risk-aware reward would change. Deep Q’s behaviour — exposure trimming that reduces drawdown at small return cost — is exactly the behaviour a reward of the form $r_t = \Delta W_t - \lambda D_t$ (drawdown-penalised) would *explicitly* pay for. Under the current risk-neutral reward, that behaviour survives only where the value function finds it locally profitable; under a risk-aware reward it would be globally incentivised. This is the single highest-leverage extension (Chapter 6).

Scope. Single asset; daily bars in monthly episodes; three seeds; a risk-neutral reward; PPO from a reference library rather than from scratch. Consequences and remedies are discussed in Chapter 6.

Chapter summary (plain language). Both home-built learners passed their video-game driving test, so their behaviour on markets can be trusted to reflect the problem, not bugs. On real market data the policy-gradient learner concluded that the best plan was to buy immediately and hold — exactly matching the passive strategy, trade for trade — and the mathematics says that conclusion is *correct* for the reward it was given, since on average the market drifted upward. The value-based learner found something subtler: it stayed almost fully invested but sold down during turbulent stretches, losing about \$2 a month of profit in exchange for noticeably smaller worst-case losses — and when trading fees were raised, it traded less, all of its own accord. The modern PPO method landed on one of two sensible answers depending on its random starting conditions. Finally, a first experiment with a reward that punishes losses directly showed the value-based learner cutting its worst losses four-fold — while the policy-gradient learner simply switched from “always fully invested” to “never invest at all.” None of this “beats the market” — and the study was designed so that saying so honestly is its central strength. What these findings mean, and what they do not, is the subject of the final chapter.

Chapter 6

Conclusions & Future Work

6.1 Achievements against objectives

All five objectives of Section 1.3 were met. **O1:** the trading MDP was formulated with a complete 9-feature observable state, a divisible-asset Discrete(3) action model with feasibility masking, and a fee-inclusive wealth-change reward whose telescoping property ($\sum r_t = W_T - W_0$) was verified numerically in every episode. **O2:** both learning rules were derived from first principles — including the likelihood-ratio derivation explaining the $\log \pi_\theta$ term — and implemented from scratch in PyTorch with no RL library in the training path. **O3:** both implementations were validated on CartPole, Flappy Bird and LunarLander, clearing random floors on all three. **O4:** both were evaluated on 26 held-out months of SPY data against do-nothing, random and fully invested buy-and-hold baselines, under a chronological split, matched 80k-step budgets and three seeds; a PPO row and a three-level fee sensitivity study completed the ablation. **O5:** the observed behaviours were analysed mechanistically, and the limitations below are stated openly.

6.2 Principal findings

1. **Under a risk-neutral wealth reward, the rational policy is buy-and-hold — and REINFORCE finds exactly that.** In the pilot (minimal state) three different policy-gradient optimisers produced action sequences identical to fully invested buy-and-hold on all 26 test months; with the complete 9-feature state, REINFORCE still converged to the identical-to-the-cent buy-and-hold policy. The collapse is a property of the *objective*, not of any optimiser or of the state alone. This is the empirical face of an old theoretical point [4]: the reward function defines the incentive, and an agent can only be blamed for optimising what it was asked to optimise.
2. **The algorithm families respond differently to the same incentive.** Deep Q-learning, trained on the same MDP, budget and data, learned a near-fully-invested policy that trades around local turbulence: lower monthly volatility (274 vs. 310), lower maximum drawdown (4.8% vs. 6.0%) and higher monthly Sharpe (0.45 vs. 0.40) than buy-and-hold, at a mean cost of \$1.96/month — and it adapted its trading intensity to transaction fees (19.8 trades/month at zero fee, 6.1 at 20 bps). PPO, whose clipping deliberately limits policy movement, was bimodal across seeds: exact buy-and-hold on two seeds, a cautious low-exposure policy on the third.
3. **Evaluation design is a result in itself.** Without the properly capitalised buy-and-hold baseline,

every learned agent in this dissertation would have been reported as beating the market by an order of magnitude — the precise failure mode the financial machine-learning methodology literature warns of [8, 26]. With it, the honest conclusion is recoverable.

6.3 Critical appraisal and limitations

What is limited. (i) A single asset and a single market regime window; conclusions about the 2022–2024 test period need not transfer to other assets or eras. (ii) Daily bars in monthly episodes rather than the intraday $T=390$ horizon originally envisaged; the environment API is horizon-agnostic, but the empirical claims hold only at the tested granularity. (iii) Three seeds bound, but do not eliminate, run-to-run variance — and the PPO bimodality shows that seed sensitivity is a real phenomenon here, not a formality. (iv) Episodes are treated as independent although consecutive months are correlated (non-IID); the MDP framework used here has no mechanism for that temporal structure. (v) The state, however deliberately designed, remains a partial observation of the market: the problem is strictly a POMDP [29], and no belief-state or recurrent machinery was used. (vi) Execution is idealised (fills at the close, unlimited liquidity at the quoted price); for SPY at these trade sizes the idealisation is mild, but it is an idealisation.

What could have been done differently. With hindsight, two choices would change. First, the pilot’s whole-share action model was a foreseeable dead end — the divisibility problem it created (fixed actions becoming infeasible as prices moved) consumed implementation time that the final divisible-slice model dissolved immediately. Second, the risk-neutral reward could have been paired from the outset with a risk-aware variant as a designed experiment; the project discovered the right question (“what happens to Deep Q’s trimming behaviour when the reward pays for it?”) only after the ablation results arrived, leaving it as future work rather than a delivered chapter.

What the project got right, in hindsight. Validating implementations on games before markets caught errors where they were diagnosable; the fully invested baseline converted a would-be false positive into the central finding; fee-inclusive rewards made the fee sensitivity study possible at all; and deriving the losses before implementing them was, per the supervision process, the difference between using an algorithm and understanding one.

6.4 Future work

- **Risk-aware rewards.** Replacing $r_t = \Delta W_t$ with a drawdown-penalised variant or the differential Sharpe ratio [4] changes the incentive that currently makes buy-and-hold optimal. The preliminary run of Section 5.6 already shows the direction: at $\lambda = 0.25$, Deep Q’s drawdown falls four-fold (6.0% to 1.4%) with Sharpe rising to 0.68, while REINFORCE flips to the all-cash corner. The full study is a λ sweep tracing the return-risk frontier each algorithm can reach, with multiple seeds.
- **Intraday horizons.** Minute-bar episodes ($T=390$) via the same environment API, testing whether the family-level behavioural differences persist at higher frequency, where fee pressure and state dynamics are sharper.

- **Partial observability.** Recurrent policies, or an explicit regime model in the spirit of hidden Markov models [30], to address the POMDP structure acknowledged throughout.
- **Continuous actions and portfolios.** Replacing the fixed trade slice with a continuous allocation, and extending to multi-asset weight vectors with risk-adjusted objectives [27, 23].
- **Value-based refinements.** Double Q-learning [18] to control the overestimation bias that masked TD targets partially mitigate here.

6.5 Reflection on project management

The project followed a staged plan agreed in supervision, summarised in Table 6.1. Two properties of the plan proved decisive. First, *stage gates with demonstrable artefacts*: no stage began until the previous one produced something checkable (passing tests, learning curves, result tables), which localised every fault to the stage that introduced it. Second, *descopeing as a deliberate act*: the most consequential replanning decision was narrowing an early multi-component ambition (uncertainty-aware portfolio management with a forecasting subsystem) to a single, fully understood trading MDP with two from-scratch algorithms. The decision was made with the supervisor, explicitly trading breadth of feature count for depth of understanding — a trade this dissertation’s examinable content vindicates.

Table 6.1: Project stages and their exit artefacts.

Period (2026)	Stage	Exit artefact
Feb–Apr	Foundations & literature	working notes; supervised derivation exercises; scope agreed
May–Jun	Phase 0: games	from-scratch REINFORCE/DQN clearing random floors on three games; suite metrics
Jun–Jul	Pilot trading MDP	minimal-state environment + tests; the buy-and-hold collapse result and fair-baseline lesson
late Jul	Supervision checkpoint	recorded feedback; corrected objective derivation set as homework; final-model scope fixed
Aug	Final model & ablation	9-feature environment (8/8 tests), three-algorithm ablation, fee study, all charts
Aug–Sep	Write-up	this document, written methodology-first as advised; abstract last

Risk management was handled by construction rather than by contingency tables: the riskiest claim in any learning project — “the implementation is correct” — was retired early by the game-validation gate; the riskiest scientific claim — “the agent beats the market” — was constrained by baselines designed to make it hard to assert falsely. Weekly supervision meetings served as checkpoints, and the final experiment suite was engineered to be regenerable in minutes on a laptop CPU, so that every claim in this dissertation can be demonstrated live at the viva.

Closing summary (plain language). Two learning programs were built from nothing, taught to play games, and then set loose on seven years of real market prices with imaginary money. One concluded that the best strategy was the simplest: buy immediately and hold — which, for the goal it was given, is

mathematically the right answer. The other learned to step partially out of the market when conditions turned rough, accepting slightly smaller profits for noticeably smaller losses, and learned to trade less when trading cost more. Neither beat the market, and the project was designed so that this honest answer would be detectable — because in this field, the experiments that claim otherwise are usually the ones with the mistake.

Appendix A

Equation-to-Implementation Traceability

A recurring requirement of this project’s supervision was that the implementation *be* the formulation: every equation in Chapter 3 must correspond to identifiable lines of code, and every claimed property must be verified by a test. This appendix makes that correspondence explicit. File paths are relative to the project repository (`experiments/`).

Why the log appears — one-paragraph summary for the viva. The objective $J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [G(\tau)]$ depends on θ only through the sampling distribution. Its gradient, $\int \nabla_\theta p_\theta(\tau) G(\tau) d\tau$, is not an expectation and cannot be estimated from rollouts — until the identity $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$ rewrites it as $\mathbb{E}_\tau [\nabla_\theta \log p_\theta(\tau) G(\tau)]$. Taking the log of the trajectory probability factorises it into a sum in which the unknown market dynamics are additive constants with respect to θ and vanish under the gradient, leaving only $\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)$. Every step is exact; the only approximation is replacing the expectation by an average over N sampled trajectories. The implemented loss is the negative of that average, because PyTorch minimises.

Table A.1: Formulation $\rightarrow$ code $\rightarrow$ verification map.

Equation / definition (Ch. 3)	Implementation	Verification
State $s_t \in \mathbb{R}^9$ (Table 3.4)	<code>env_full.py</code> , <code>_obs()</code> : nine feature methods	shape/clock tests; feature ranges inspected per episode
Buy/Sell of slice Δ with fee c (Sec. 3.4)	<code>env_full.py</code> , <code>_buy()/_sell()</code>	exact fee-charge test on a flat price path
Feasibility mask; forced Hold on illegal action	<code>env_full.py</code> , <code>action_masks()</code> , <code>step()</code>	mask tests (Sell-while-flat blocked; Buy blocked at zero cash)
Mandatory terminal liquidation	<code>env_full.py</code> , <code>step()</code> terminal branch	test: zero units and positive cash at episode end
Reward $r_t = W_{t+1} - W_t$, Eq. (3.4); telescoping $\sum_t r_t = W_T - W_0$	<code>env_full.py</code> : reward computed from wealth difference	accounting-identity test under random play; re-checked in <i>every</i> evaluated episode ($< 10^{-6}$)
Rewards-to-go G_t (Step 4, Sec. 3.7)	<code>reinforce_trading.py</code> : reversed discounted accumulation	spot-checked against closed form on short synthetic episodes
REINFORCE loss $L(\theta) = -\sum_t \log \pi_\theta(a_t s_t) G_t$, Eq. (3.14)	<code>reinforce_trading.py</code> : <code>loss = -(log_probs * returns).sum()</code>	Phase-0 validation: clears random floor on three games
Masked categorical $\pi_\theta(\cdot s)$ (Sec. 3.4)	<code>reinforce_trading.py</code> , <code>masked_dist()</code> : illegal logits $\rightarrow -10^9$ before softmax	illegal-proposal rate = 0 by construction for scratch methods
TD target $y = r + \gamma(1-d) \max_{a' \text{ legal}} Q_{\phi^-}(s', a')$, Eq. (3.16)	<code>dqn_trading.py</code> : target net + stored next-state mask applied before the max	Phase-0 validation; masked-target inspection on buffer samples
Replay buffer, target sync, ϵ -greedy anneal (Sec. 3.8)	<code>dqn_trading.py</code> : <code>deque(50k)</code> , sync every 500 steps, linear $\epsilon : 1.0 \rightarrow 0.05$	training curves stabilise (Fig. 5.6)
PPO clipped surrogate, Eq. (3.18)	Stable-Baselines3 PPO (reference implementation)	benchmarked library; protocol asymmetry documented in Sec. 4.6
Chronological split, no shuffling (Sec. 3.11)	<code>fetch_spy_daily.py</code> : month grouping, 70/30 cut in time order	split metadata (dates, IDs) persisted in JSON alongside results

Appendix B

Per-Month Test Results

Wealth change ΔW in dollars for every held-out test month and every method (seed 42; $C_0 = \$10,000$; fee 5 bps). Generated directly from `full_ablation_results.json`; B0 do-nothing is identically zero and omitted. Months where REINFORCE equals buy-and-hold to the cent reflect identical executed action sequences.

Table B.1: Per-month ΔW (\$) on the 26 held-out test months.

Month	B1b buy&hold	REINFORCE	Deep Q	PPO	Random
2022-11	636.79	636.79	645.75	243.35	233.11
2022-12	-381.15	-381.15	-299.17	-154.18	26.35
2023-01	417.34	417.34	425.28	194.82	83.54
2023-02	-382.32	-382.32	-230.41	-126.52	-59.94
2023-03	388.38	388.38	301.21	87.75	200.79
2023-04	125.73	125.73	116.98	47.27	-34.12
2023-05	151.10	151.10	123.44	43.74	46.37
2023-06	342.53	342.53	319.81	133.13	157.41
2023-07	294.37	294.37	267.37	102.76	3.13
2023-08	26.90	26.90	-80.74	-12.57	22.29
2023-09	-419.31	-419.31	-286.25	-135.77	-72.98
2023-10	-273.74	-273.74	-198.85	-47.53	3.10
2023-11	456.48	456.48	422.93	186.11	66.66
2023-12	333.98	333.98	313.55	126.57	47.29
2024-01	200.01	200.01	170.11	82.00	66.03
2024-02	231.14	231.14	193.81	94.80	8.43
2024-03	222.86	222.86	189.73	78.75	118.77
2024-04	-305.31	-305.31	-308.99	-107.59	-101.08
2024-05	226.38	226.38	211.84	126.56	61.49
2024-06	179.23	179.23	163.28	85.78	33.37
2024-07	-86.15	-86.15	-112.32	9.12	-28.66
2024-08	594.48	594.48	537.46	182.13	5.01
2024-09	404.85	404.85	386.29	131.06	69.32
2024-10	-96.25	-96.25	-73.17	-1.91	-54.59
2024-11	225.92	225.92	233.51	154.31	100.80
2024-12	-259.37	-259.37	-228.47	-76.61	-14.65
Mean	125.19	125.19	123.23	55.67	37.97

Appendix C

Glossary (Plain-Language)

This glossary explains the recurring technical terms of the dissertation in everyday language. Definitions are intentionally informal; the precise versions live in Chapter 3.

Agent The learning program — the “player.” Here: a small neural network deciding each day whether to hold, buy, or sell.

Environment The world the agent acts in. Here: a simulated trading month built from real historical prices, which executes the agent’s trades, charges fees, and keeps the books.

State Everything the agent is shown before deciding — its “eyes.” Here: nine numbers covering recent market behaviour, how far through the month it is, and its own cash and holdings.

Action One of the choices available. Here there are exactly three: do nothing, buy a \$1,000 slice of the asset, or sell one.

Reward The score received after each action. Here: the day’s change in total wealth, with trading fees already deducted. The agent’s whole purpose is to make the month’s total score as large as possible.

Episode One complete round of the task, after which everything resets. Here: one calendar month of trading, ending with a forced sale of any remaining holdings.

Policy The agent’s decision rule — its “habits.” Formally a function from state to action probabilities; informally, what it would do in any situation.

Policy gradient / REINFORCE A family of methods that improve the policy directly: after each month, decisions that preceded a good outcome are made more probable, and vice versa. REINFORCE is the original, simplest member.

Q-value / Deep Q-learning The other family: learn a “price list” saying how much future profit each action is worth in each situation, then take the best-priced action. “Deep” means the price list is a neural network rather than a lookup table.

PPO A modern, stabilised descendant of REINFORCE that limits how much the decision rule can change per update — steadier, but capable of settling into whichever behaviour it found first.

Neural network A flexible mathematical function with many adjustable dials (parameters), tuned gradually so its outputs improve against an objective.

Training / test split The data is cut in two by date: the agent learns only on the earlier part (2018–Oct 2022) and is examined only on the later part (Nov 2022–Dec 2024), which it has never seen — like a mock exam written after the revision period ended.

Baseline A simple non-learning strategy every agent must be compared against. The important one here is *buy-and-hold*: invest everything at the start of the month and wait.

Buy-and-hold The classic passive strategy. Decades of evidence say it is genuinely hard to beat using price history alone — which is why beating it easily in an experiment is grounds for suspicion, not celebration.

Transaction fee / basis points (bps) The cost of trading, charged as a fraction of the traded amount. 5 bps = 0.05%, i.e. 50 cents per \$1,000 traded.

Sharpe ratio Profit earned per unit of ups-and-downs endured. Higher is better, but a strategy that barely invests can post a flattering Sharpe simply by barely moving.

Maximum drawdown The worst peak-to-trough fall an investor following the strategy would have suffered — a measure of the most painful stretch, not the average one.

Exposure How much of the capital is invested in the asset at a given moment, from 0% (all cash) to 100% (fully invested).

Seed The starting value of the random number generator. Running the same experiment with several seeds checks that a result is not a fluke of one lucky roll.

Overfitting Learning the quirks of the practice data rather than the lesson, and therefore failing on new data. The chronological split and held-out months guard against it.

Masking Greying-out impossible actions (selling what you do not own; buying with no cash) so the agent can neither attempt nor learn from illegal moves.

Accounting identity The bookkeeping check that the sum of all daily scores in a month exactly equals the month's true profit. It is verified in every single experiment; if it ever failed, the simulator would be lying to the agent.

Markov property The requirement that the state contain everything relevant from the past — like a chess photograph being enough to resume the game. Much of this project's design effort went into honouring it.

Partial observability (POMDP) The honest admission that no price chart contains everything that moves markets (news, order flow, other traders' intentions). The theory for acting under such hidden information exists but is much harder; here it is acknowledged as a limitation and a future direction.

Bibliography

- [1] G. Tesauro, “Temporal difference learning and TD-Gammon,” *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [4] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Transactions on Neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.
- [5] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, “Deep direct reinforcement learning for financial signal representation and trading,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 3, pp. 653–664, 2017.
- [6] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, “FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance,” *arXiv preprint arXiv:2011.09607*, 2020.
- [7] E. F. Fama, “Efficient capital markets: A review of theory and empirical work,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [8] D. H. Bailey, J. M. Borwein, M. L. de Prado, and Q. J. Zhu, “Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance,” *Notices of the American Mathematical Society*, vol. 61, no. 5, pp. 458–471, 2014.
- [9] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [13] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [14] R. S. Sutton, “Learning to predict by the methods of temporal differences,” *Machine Learning*, vol. 3, no. 1, pp. 9–44, 1988.
- [15] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [16] J. N. Tsitsiklis and B. Van Roy, “An analysis of temporal-difference learning with function approximation,” *IEEE Transactions on Automatic Control*, vol. 42, no. 5, pp. 674–690, 1997.
- [17] L.-J. Lin, “Self-improving reactive agents based on reinforcement learning, planning and teaching,” *Machine Learning*, vol. 8, no. 3–4, pp. 293–321, 1992.
- [18] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.
- [19] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems 12*, 2000, pp. 1008–1014.
- [20] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016, pp. 1928–1937.
- [21] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016.
- [22] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 1889–1897.
- [23] Z. Jiang, D. Xu, and J. Liang, “A deep reinforcement learning framework for the financial portfolio management problem,” *arXiv preprint arXiv:1706.10059*, 2017.
- [24] T. Théate and D. Ernst, “An application of deep reinforcement learning to algorithmic trading,” *Expert Systems with Applications*, vol. 173, p. 114632, 2021.
- [25] T. G. Fischer, “Reinforcement learning in financial markets — a survey,” FAU Discussion Papers in Economics, Friedrich-Alexander University Erlangen-Nürnberg, Tech. Rep. 12/2018, 2018.
- [26] M. L. de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- [27] H. Markowitz, “Portfolio selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

- [28] W. F. Sharpe, “The Sharpe ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.
- [29] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [30] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [31] J. W. Wilder, *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research, 1978.
- [32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
- [33] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- [34] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “OpenAI Gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [35] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.